

Universidad Nacional de Cuyo

Facultad de Ingeniería

Lic. en Ciencias de la Computación

Licenciatura en Ciencias de la Computación

Compiladores
Manual del Desarrollador de TinyRust+

Rocío Belén Corral

Universidad Nacional de Cuyo

Facultad de Ingeniería

Mayo 2023

Introducción **3**
Primeros pasos **5**
Instalación 5
Compilación 5
Ejecución 6
Pruebas 7
Arquitectura **8**
Punto de entrada 11
Manejo de errores 12
Análisis léxico **13**
Alfabeto de entrada 13
Palabras reservadas 13
Tokens 13
Errores 16
Implementación 17
Executor 17
LexicAnalyzer 17
Token 20
ReservedWords 20
Análisis sintáctico **21**
Gramática 21
BNF Extendida 21
Gramática final 24
Primeros 26
Siguientes 29
Errores 29
Implementación 30
Executor 30
SyntacticAnalyzer 30
Análisis semántico **32**
Esquema de traducción 33
Validaciones 38
Errores 39
Validaciones semánticas 40
Implementación 43
Executor 43
SymbolTable 44
Entidades de la Tabla de Símbolos 46
ClassEntry 46
Generación de código **55**
Arquitectura 55
Representación de clases y polimorfismo 55
Registro de activación 56

Implementación..... 57
AbstractSyntaxTree..... 57
MethodCallNode..... 57
MethodNode..... 58
SentenceNode..... 58
ExpressionNode..... 58
Alcance actual y futuros cambios..... 59
Ensamblado y ejecución..... 59
Ejemplo..... 62

Introducción

TinyRust+ es un lenguaje de programación multipropósito orientado a objetos, type safe y con sintaxis similar al lenguaje Rust. Es un lenguaje reducido, pero igualmente incluye características propias de los lenguajes de programación modernos. TinyRust+ es compilado y fuertemente tipado, por lo que impone una disciplina fuerte de programación y garantiza que no ocurran errores de tipo en tiempo de ejecución.

El objetivo de este manual es explicar la arquitectura y funcionamiento interno del compilador de este lenguaje para facilitar su extensión con nuevas funcionalidades en un futuro.

La implementación del compilador está separada según las etapas de análisis de los archivos de entrada: análisis léxico, análisis sintáctico y análisis semántico. Por último, la generación de código se incluye en las estructuras generadas durante la etapa de análisis semántico.

La primera etapa de análisis es llevada a cabo por el analizador léxico, programa que genera un conjunto de *tokens* a partir de un archivo de texto. Estos tokens se definen utilizando como base las reglas sintácticas de TinyRust+ para delimitar palabras reservadas, operadores, identificadores, tipos y demás.

La segunda etapa es protagonizada por el analizador sintáctico, programa que consume los *tokens* generados por el analizador léxico y verifica que el programa cumpla con las reglas de la gramática de TinyRust+. Esta implementación consiste en un analizador sintáctico descendente predictivo recursivo. Antes de su implementación, se tuvo que tratar la gramática de TinyRust+ en formato EBNF transformándola a notación BNF, eliminando la recursividad a izquierda y factorizando.

Para la tercera etapa, donde se realiza el análisis semántico, construye y se hace uso de dos estructuras: la tabla de símbolos y el árbol sintáctico abstracto).

La tabla de símbolos (TS, por sus siglas abreviadas) se construye de forma intercalada a medida que se realiza el análisis sintáctico del archivo de entrada. Durante el análisis sintáctico se agregan entidades a la tabla y para cuando se ha completado el análisis exitosamente, la tabla de símbolos está completa y se realiza su consolidación.

El árbol sintáctico abstracto (AST, por sus siglas en inglés) se construye de forma recursiva en simultáneo con el análisis sintáctico y la construcción de la tabla de símbolos. Para cuando se ha completado la construcción del AST, se realizan sobre él las validaciones de sentencias del archivo.

En la última etapa, tras un análisis exitoso, se genera código de salida ASM a partir de la información contenida en el árbol sintáctico abstracto y la tabla de símbolos.

En este manual se abordará la implementación de cada uno de estos analizadores por separado y se explicará cómo interactúan entre sí dentro del sistema para lograr generar un

archivo ASM con código MIPS32 ejecutable que tenga el comportamiento exacto que se espera para el archivo de entrada en formato .rs según la definición del lenguaje TinyRust+.

Primeros pasos

Un código fuente a compilar en TinyRust+ consiste de un único archivo, ya que el lenguaje no posee un sistema de importado/exportado de dependencias. El archivo debe tener extensión *.rs*. El siguiente fragmento de código muestra el ejemplo más básico de código en TinyRust+.

```
// holaMundo.rs

fn main() {
  (IO.out_str("Hola mundo!"));
}
```

Este ejemplo muestra por pantalla la cadena “Hola mundo!” y sirve como un primer ejemplo para probar el correcto funcionamiento del lenguaje.

Vale la pena aclarar que aunque la extensión *.rs* coincide con la del lenguaje Rust, los archivos de TinyRust+ no tienen exactamente la misma sintaxis. Por lo que obtendrá un error si intenta compilarlos con un compilador de Rust.

Instalación

El compilador de TinyRust+ está implementado en Java y la versión compilada del proyecto consiste en un archivo portable en formato *JAR*[1], por lo que requiere tener instalado Java para su funcionamiento. La versión de Java recomendada es la 17. En versiones previas no se garantiza el correcto funcionamiento del compilador. Para trabajar sobre el código fuente del compilador no se requiere ninguna herramienta extra más que el SDK de Java 17 (o superior) y cualquier editor de código.

Compilación

Durante el desarrollo, el proyecto se compila y corre ejecutando los siguientes comandos o utilizando cualquier herramienta automatizada para compilar y ejecutar proyectos Java.

```
javac *.java
java <projectRoot>/TinyRustCompiler <ruta archivo entrada>.rs
[<ruta archivo salida]
```

En la etapa de desarrollo y pruebas, es posible especificar el nombre de un archivo de entrada de prueba dentro del código de la clase de entrada *TinyRustCompiler*, de modo que no se tenga que especificar la ruta del archivo en cada ejecución.

Habiendo compilado el proyecto, es posible armar un archivo JAR portable para distribuir a los usuarios de TinyRust+.

[1] Java Archive (JAR) Files

```
jar cfm tinyRust.jar Manifest.txt *
```

Utilizando el archivo *JAR*, un programa de TinyRust+ se compila invocando al JAR del compilador y especificando la ruta del archivo de entrada y, opcionalmente, la ruta del archivo de salida.

```
java -jar <ruta del compilador>.jar <ruta archivo entrada>.rs
[<ruta archivo salida]
```

Por ejemplo, para compilar el archivo TinyRust+ de ejemplo al principio de esta sección se podría ejecutar el siguiente comando:

```
java -jar ~/tinyRust.jar ./holaMundo.rs ./out/holaMundo
```

Si no se especificara un destino para el archivo de salida, este será generado por defecto en la misma ubicación que el archivo de entrada y con el mismo nombre, pero con extensión *ASM*[2]. Si el archivo existía previamente se sobreescribirá, caso contrario será creado por el compilador.

Por pantalla siempre siempre se muestra el resultado del proceso. Este resultado puede ser un mensaje de éxito o la información de un error, en caso de que el análisis del código no se haya podido completar correctamente. En caso de no especificarse este parámetro, la salida se mostrará por pantalla.

Solo en caso de que el análisis haya sido exitoso, se crearán también dos archivos de igual nombre que el archivo de entrada pero con distinta extensión. El archivo de extensión *.ts.json* guarda una representación en formato JSON de la tabla de símbolos con todo su contenido, mientras que el archivo con extensión *.ast.json* guarda una representación en formato JSON del árbol sintáctico abstracto del archivo. Estos archivos pueden ser de suma utilidad para el programador durante el desarrollo para validar que el resultado del análisis sea el esperado para la entrada.

El archivo de salida ASM posee código assembler MIPS32[3], por lo cual es ejecutable dentro de un simulador de esta arquitectura, como SPIM, MARS o similares. Al ejecutar este archivo en el simulador, para el caso de ejemplo, la salida por pantalla esperada sería la siguiente:

```
Hola mundo!
```

Ejecución

Para la ejecución del código ASM generado por el compilador se debe utilizar algún simulador de la arquitectura MIPS32, como SPIM, MARS o similares. En la sección

[2] Formato de archivo ASM - Formato de archivo de código fuente de lenguaje ensamblador
[3] Imagination's MIPS32 architecture

correspondiente a la generación de código de este manual se explica de forma detallada el modo de uso de una de estas herramientas.

Pruebas

Durante la etapa de desarrollo y *testing*, es posible colocar *breakpoints* en cualquier punto del código y ejecutar el proyecto en modo *debug* para seguir interactivamente el flujo de ejecución del compilador.

En el repositorio del proyecto se incluyen los casos de prueba que se utilizaron para poner a prueba el funcionamiento de cada etapa de análisis. Dichos archivos se pueden encontrar en el directorio *tests* del proyecto, donde se encuentra una carpeta por cada etapa de análisis, conteniendo archivos de prueba con casos exitosos y casos de fallo. Cada archivo incluye al final en un comentario cuál es la salida esperada, ya sea un caso de éxito o de error con su detalle.

En cada etapa, los primeros tests del conjunto son los casos exitosos, los cuales son minoría, dado que la gran mayoría de pruebas son de casos de falla. Para los tests exitosos del analizador léxico, se incluye la lista de tokens que debe generar. Para los tests exitosos del analizador semántico, se incluyen los archivos JSON de la TS y el AST del archivo.

Los test de cada etapa garantizan un análisis exitoso en etapas previas. Es decir, los tests sintácticos son válidos léxicamente. Los tests de análisis semántico de declaraciones son válidos sintácticamente y los tests de análisis semántico de sentencias tienen un análisis de declaraciones exitoso. De este modo, se pueden probar los errores y casos de éxito esperados en cada etapa de forma aislada.

Arquitectura

El repositorio del proyecto se encuentra estructurado en una jerarquía de directorios para facilitar su legibilidad. En la **Figura 1** puede apreciarse esta jerarquía.

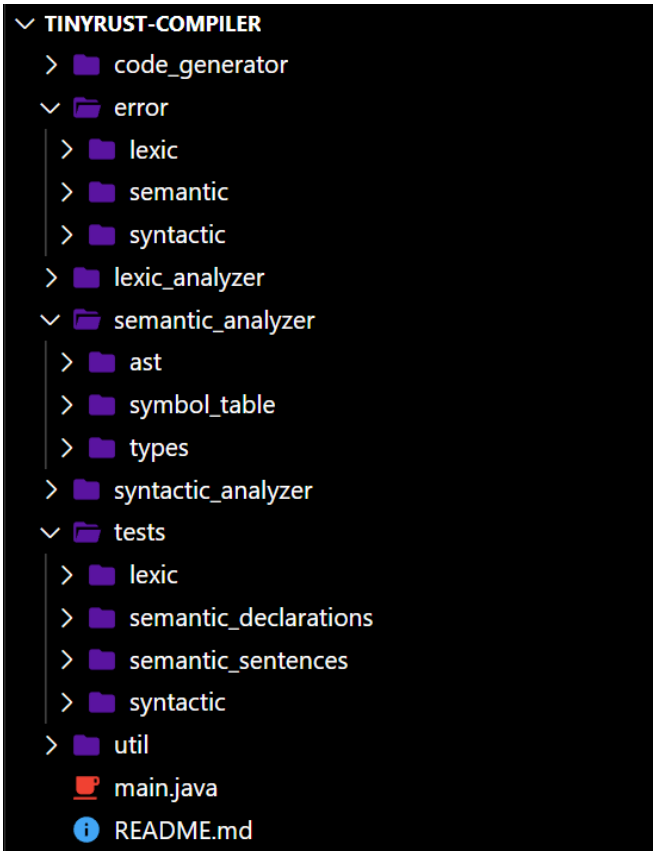


Figura 1. Jerarquía de carpetas

En el directorio raíz se encuentran los siguientes archivos y directorios:

1. Archivo *main.java*: contiene la clase TinyRustCompiler, el punto de entrada del compilador.

2. Carpeta *util*: contiene clases que proveen algún tipo de asistencia mediante métodos. Por ejemplo: la clase Logger (provee métodos necesarios para escribir archivos de salida), la clase Json (provee métodos para generación de archivos en formato JSON[2]), entre otras.

3. Carpeta *tests*: contiene los archivos *.rs* de prueba utilizados para probar el compilador. Contiene una carpeta por cada etapa de análisis con archivos de prueba para esa etapa. Cada archivo de prueba incluye el output esperado al analizarlo.

4. Carpeta *lexic_analyzer:* contiene todas las clases relacionadas al analizador léxico del compilador. Incluye al analizador léxico, un ejecutor y las clases *Token* y *ReservedWords*.

5. Carpeta *syntactic_analyzer*: contiene todas las clases relacionadas al analizador sintáctico del compilador. Incluye al analizador sintáctico y un ejecutor.

6. Carpeta *semantic_analyzer*: incluye una subcarpeta para clases relacionadas con el AST, otra para la TS y una conteniendo tipos compartidos por ambas partes. En el caso de la TS se incluyen clases para cada tipo de entidad de la tabla y la clase que representa a la tabla de símbolos propiamente. En el caso del AST se incluyen clases para cada tipo de nodo del árbol y la clase que representa el nodo raíz del árbol. En la última carpeta se incluye una clase para cada tipo de dato de TinyRust+, que luego se utilizan tanto en la TS como en el AST. Se incluye un ejecutor para únicamente la TS y otro para la TS y el AST en conjunto.

7. Carpeta *code_generator*: contiene un ejecutor que realiza el proceso completo de ejecución y generación de código.

8. Carpeta *error*: contiene una clase para cada posible error que pueda ser arrojado al momento de ejecutar el compilador de TinyRust+. Contiene una carpeta por cada etapa de análisis, donde se encuentra una clase para cada error que pueda aparecer en esa etapa.

En las **Figuras 2**, **3** y **4** se encuentra seccionado el diagrama UML de clases completo del proyecto. Las clases que se repiten entre un diagrama y otro (Logger, TinyRustCompiler, SyntacticAnalyzer) son el punto de conexión entre ambos. La división en múltiples figuras se realizó debido a la dimensión del proyecto para facilitar su legibilidad.

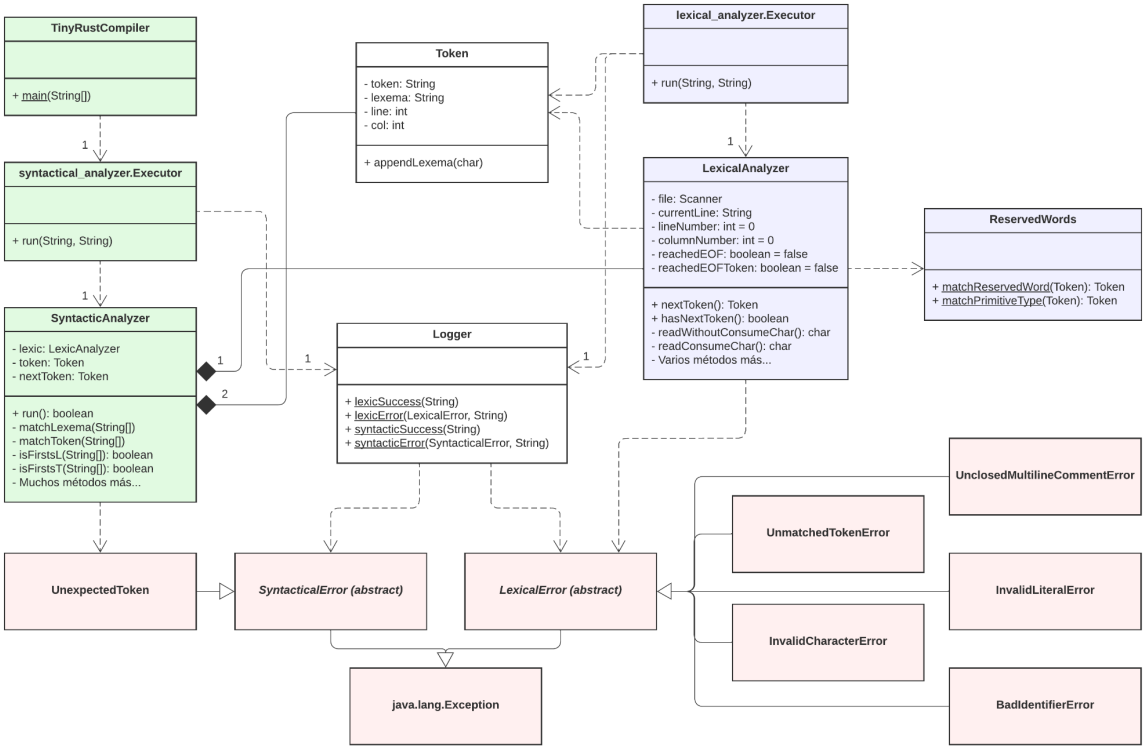


Figura 2. Diagrama de clases UML de los analizadores léxico y sintáctico de TinyRust+.

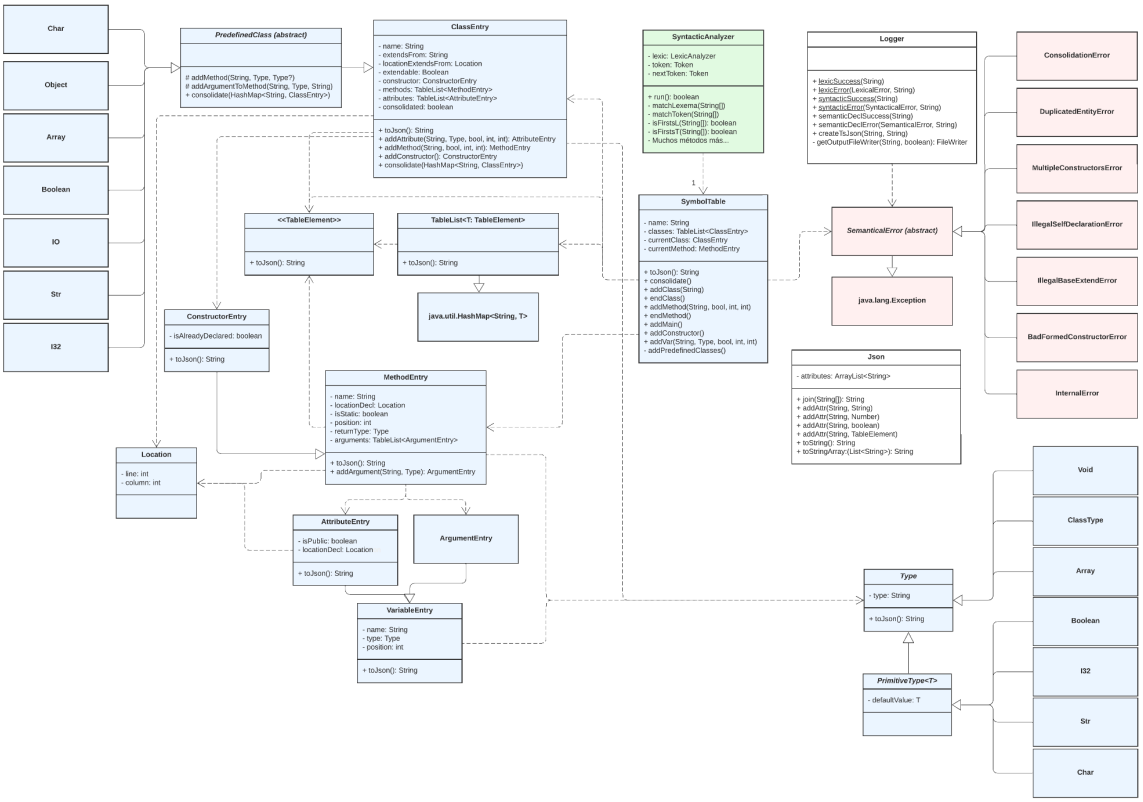


Figura 3. Diagrama de clases UML de la tabla de símbolos de TinyRust+.

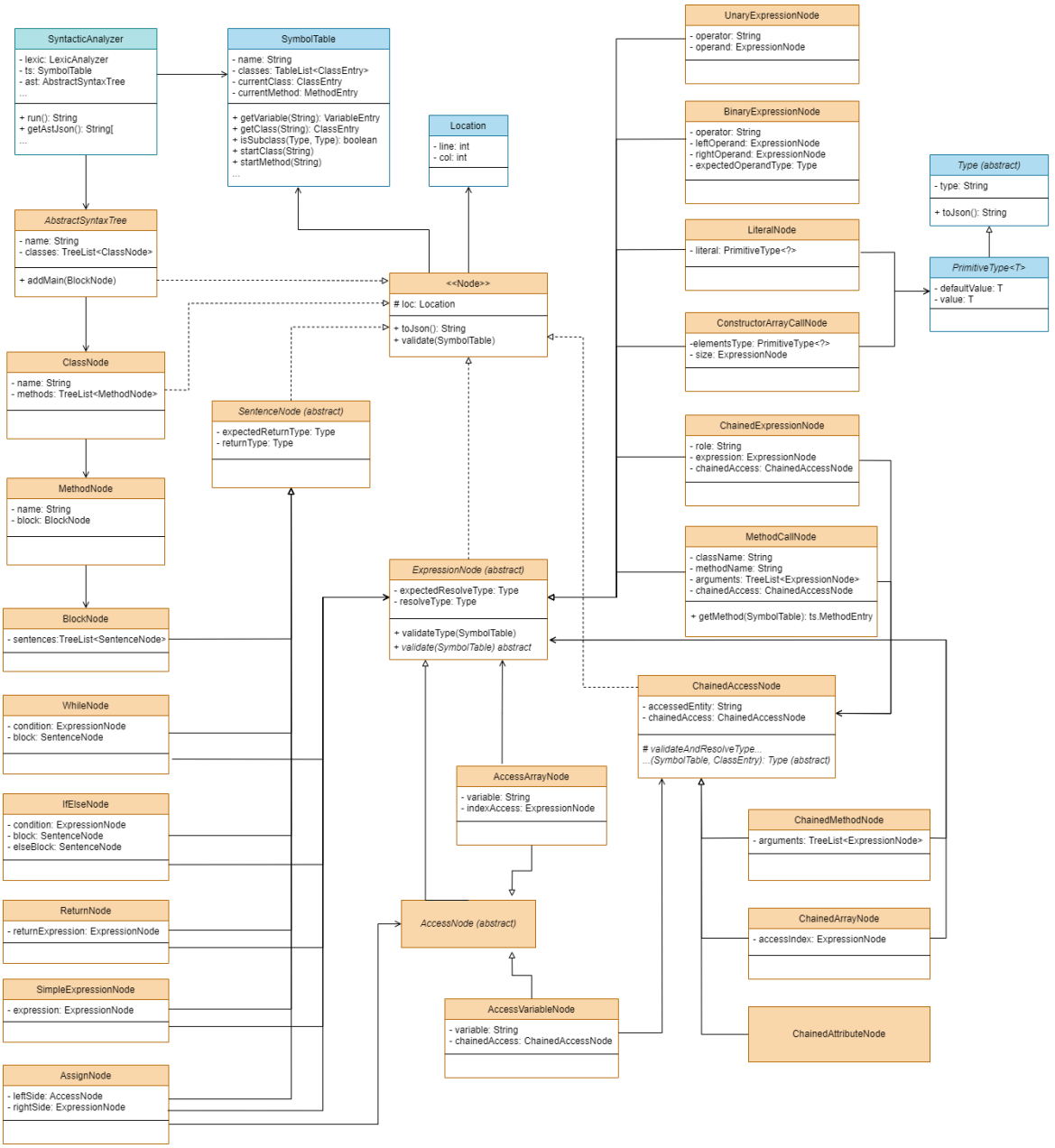


Figura 4. Diagrama de clases del AST de TinyRust+.

Punto de entrada

En el archivo main.java se encuentra declarada la clase *TinyRustCompiler*, la clase raíz del compilador. El método main de esta clase es el punto de entrada de la ejecución del compilador de TinyRust+. Cuando comienza la ejecución, en esta clase se leen los argumentos de la línea de comandos para los archivos de entrada y salida y se guardan. En el método main de esta clase se instancia la clase *Executor* y se invoca su método *run* pasándole los nombres de los archivos de entrada y salida para comenzar el proceso de compilación..

Los errores relacionados con los archivos de entrada o salida se manejan en esta clase, mostrando un mensaje de error acorde y deteniendo la ejecución del proceso.

La clase *Executor* responde a una interfaz intercambiable. Para cada etapa de análisis se

implementó una clase *Executor* distinta que se encargue de invocar al analizador exclusivamente de esa etapa, de modo que se realizará el análisis de esa etapa y las previas, pero no las posteriores. Esto sirve para poder probar fácilmente de forma aislada el comportamiento de cada etapa. En caso de querer realizar el análisis completo y la generación de código, debe instanciarse el *Executor* de la generación de código.

Manejo de errores

Para el manejo de errores de cada etapa de análisis se creó una clase abstracta distinta que hereda de la clase *Exception* de Java. Los errores lanzados por el analizador léxico heredan de la clase abstracta *LexicalError*, mientras que los errores del analizador sintáctico heredan de *SyntacticalError* y los errores semánticos heredan de *SemanticalError*. Una nueva instancia de cualquiera de estas clases de excepción espera recibir el número de línea y columna donde ocurrió el error junto con una descripción. A la hora de mostrar el mensaje de error al usuario, se incluirán estos tres datos junto con el nombre de la etapa en que ocurrió el error.

Dentro del *Executor* deben atraparse todos los errores de análisis que puedan saltar en las distintas etapas ejecutadas, ya que estos no deben llegar a la clase superior *TinyRustCompiler*. TinyRust+ no implementa técnicas de recuperación de errores, por lo cual la estrategia ante la ocurrencia de cualquier error es comunicar el error al usuario y abortar la ejecución del compilador.

Análisis léxico

El analizador léxico es un programa que genera un conjunto de *tokens* a partir de un archivo de texto. Estos tokens se definen utilizando como base las reglas sintácticas de TinyRust+ para delimitar palabras reservadas, operadores, identificadores, tipos y demás.

Para entender la implementación del analizador léxico, primero se deben tener en cuenta ciertos aspectos técnicos del lenguaje como el alfabeto de entrada, palabras reservadas y tipos de token.

Alfabeto de entrada

El alfabeto de entrada del compilador consta de los caracteres con código ASCII desde 32 hasta 126[4]. Adicionalmente, también se incluyen los caracteres:

- Salto de línea (ASCII 10)
- Tabulación (ASCII 9)
- Fin de texto (ASCII 3)
- Tabulación vertical (ASCII 11)

En la Tabla 1 se encuentra definido por extensión el alfabeto de TinyRust+.

[space]	!	"	#	$	%	&	'	(	)	*	+	,	-	.	0	1	2	3	4
[tab]	5	6	7	8	9	:	;	<	=	>	?	@	A	B	C	D	E	F	G
[vtab]	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
[break]	[	]	_	`	a	b	c	d	e	f	g	h	i	j	k	l	m	n	o
[etx]	p	q	r	s	t	u	v	w	x	y	z	{	|	}	~	\	/	^	

Tabla 1. Alfabeto de entrada de TinyRust+.

Cabe destacar que ciertos caracteres exclusivos del español, como “ñ” y vocales con tilde (tanto mayúsculas como minúsculas) no forman parte del alfabeto de entrada de este compilador.

Palabras reservadas

El lenguaje TinyRust+ posee una serie de palabras utilizadas para el control de flujo que son tratadas de forma especial. Estas son las palabras reservadas y cada una es interpretada con un token especial de ese tipo de palabra.

Tokens

En las tablas 2 y 3 se puede apreciar la lista de tokens que el analizador léxico

[4] ASCII Code. ASCII table according to Windows-1252

reconoce. En cada tabla se encuentra el tipo del token acompañado de la expresión regular que lo representa. La Tabla 2 muestra la lista de tokens de palabras reservadas del lenguaje, mientras que la Tabla 3 contiene todos los demás tokens reconocidos tales como literales, operadores, comentarios y delimitadores.

Tipo de token	Expresión regular
p_class	class
p_else	else
p_false	false
p_if	if
p_return	return
p_while	while
p_true	true
p_nil	nil
p_false	false
p_new	new
p_func	fn
p_static	static
p_pub	pub
p_void	void

Tabla 2. Tokens de palabras reservadas.

Tipo de token	Expresión Regular	Notas
id	(a..z + _)(a..z + A..Z + _ + 0..9)*	Identificador: comienzan con una letra minúscula o guión bajo y luego pueden contener cualquier cantidad de letras minúsculas, mayúsculas, números o guiones bajos. Exceptuando las palabras reservadas.
lit_int	$(0..9)^+$	Literal entero: cualquier sucesión de dígitos numéricos.
lit_string	“Σ*”	Literal cadena: contiene cualquier cantidad de caracteres del alfabeto de entrada encerrados entre comillas dobles.
lit_char	‘(Σ + \Σ)’	Literal caracter: contiene posiblemente un

		backslash y luego un único caracter del alfabeto de entrada encerrado entre comillas simples.
type	(A..Z)(a..z + A..Z + _ + 0..9)*	Tipo de dato: similar a un identificador, solo que comienza con una letra mayúscula.
multi_line_comment	/* Σ* */	Comentario multilínea: puede contener cualquier cantidad y tipo de caracteres (incluyendo saltos de línea) encerrados entre ambas secuencias de apertura y cierre.
single_line_comment	//Σ* ([break] + [etx])	Comentario de una línea: comienza con un par de slashes y puede contener cualquier cantidad y tipo de caracteres, finalizando en un salto de línea (o final del archivo).
p_true	true	
p_false	false	
op_add	+	
op_sub	-	
op_prod	*	
op_div	/	
op_mod	%	
op_or	||	
op_and	&&	
op_less	<	
op_great	>	
op_less_eq	<=	
op_great_eq	>=	
op_equal	==	
assignment	=	

semicolon	;	
colon	:	
open_par	(	
close_par	)	
open_curly	{	
close_curly	}	
open_bracket	[	
close_bracket	]	
return_type	->	
op_not	!	
op_not_eq	!=	
comma	,	
dot	.	

Tabla 3. Tokens de literales, operadores, comentarios y delimitadores.

Errores

Los distintos tipos de errores de análisis que se generan en esta etapa son:

- *Identificador mal formado*: error generado al encontrar un lexema que es cercano a un identificador pero no cumple con su sintaxis por algún motivo. Por ejemplo, la cadena '123var' comienza con dígitos seguidos de letras, por lo cual no es ni un literal entero ni un identificador. Otro ejemplo podría ser una cadena que comienza con minúsculas y, sin un espacio de por medio, tiene un "@", por ejemplo, "vari@ble".
- *Comentario sin cerrar*: error generado al encontrar una secuencia de apertura de comentario multilínea y luego llegar al final del archivo sin encontrar la secuencia de cierre, es decir, "*/".
- *Símbolo no perteneciente al alfabeto*: error generado al leer en el archivo de entrada un caracter que no esté incluido en el alfabeto Σ, como por ejemplo: "¬", "ñ", "á", etc.
- *Token desconocido*: error generado al encontrar un lexema que no cumple ningún patrón reconocible, por ejemplo, "|". Cabe destacar que "||" sí es reconocido como el operador lógico OR. Sin embargo, "|" por sí solo, no es un token de este compilador.
- *Literal inválido*: error generado al hallar literales de tipo cadena o caracter con una

estructura inválida. Por ejemplo, literal cadena sin cerrar o literal caracter conteniendo más de un caracter.

- *Literal entero inválido*: error generado al hallar un literal numérico entero cuyo tamaño es mayor que el representable mediante 32 bits.

En la implementación se creó la clase abstracta *LexicalError* y una clase que herede de ella por cada uno de estos errores. Al lanzarse el error, se incluye una descripción detallada y línea y columna donde ocurrió.

Implementación

El punto de entrada del analizador léxico es la clase *Executor*. Para la implementación de la funcionalidad de esta etapa se usan también las clases *LexicalAnalyzer*, *ReservedWords* y *Token*.

Executor

Esta clase consta de un único método llamado *run*. Al invocar este método, se crea una instancia de la clase *LexicAnalizer* que recibe como parámetro la ruta del archivo de entrada. Una vez instanciado el analizador léxico, el ejecutor llamará iterativamente al método *nextToken* del *LexicAnalyzer*, el cual devuelve el siguiente token encontrado en el archivo de entrada. Al recibir el token, lo agrega a una lista de *Tokens,* la cual será mostrada en el archivo de salida o por pantalla en el caso de tener un análisis léxico exitoso. Caso contrario, la salida mostrará en qué línea, columna y token se presentó el error.

LexicAnalyzer

Esta clase es la encargada de todo el proceso de análisis léxico. En su constructor se abre el archivo de entrada e inicializan contadores de línea y columna.

LexicAnalyzer cuenta con diversos métodos que permiten el correcto análisis léxico de un archivo de texto. Provee únicamente dos métodos públicos, que se describen a continuación:

nextToken: consume parte del archivo de entrada para reconocer y retornar el siguiente token. En caso de haber consumido todo el archivo, devuelve el token EOF (end of file). Si ocurriera algún error al intentar decodificar el archivo, este método arrojaría un error del tipo *LexicalError.*

hasNextToken: verifica si ya se ha alcanzado el final del archivo y devuelve un valor booleano acorde. Este método puede invocarse como validación antes de invocar a *nextToken*. Siempre que *hasNextToken* devuelva *false*, está garantizado que *nextToken* devolverá el token EOF (end of file).

A su vez, *LexicAnalyzer* implementa una gran cantidad de métodos privados que son utilizados directa o indirectamente por el método *nextToken*. Los métodos privados de la clase se listan y describen a continuación.

Validadores: *isValidChar, isLowercaseChar, isUppercaseChar, isAlphabet, isDigit, isOperator, isBlankSpace, isLineBreak.* Estos métodos reciben una variable de tipo char y verifican si ese caracter pertenece a cierto conjunto en particular.

isValidChar: recibe una variable de tipo char y verifica si este caracter pertenece al alfabeto de entrada.

isLowercaseChar: recibe una variable de tipo char y verifica si se encuentra en el rango de los caracteres en minúscula.

isUppercaseChar: análogo a *isLowercaseChar*, este método recibe un caracter y determina si es mayúscula.

isAlphabet: recibe una variable char y determina si es una letra del abecedario (alfabeto español).

isDigit: recibe una variable char y comprueba si es un dígito numérico.

isOperator: recibe una variable char y determina si es un operador o algún delimitador.

isBlankSpace: recibe una variable char y comprueba si se trata de algún caracter de espacio en blanco, tales como saltos de línea, tabulaciones, espacios y final de archivo.

isLineBreak: recibe una variable char y determina si se trata de un salto de línea o final de archivo.

Matchers: *matchIntLiteral*, *matchStringLiteral, matchCharLiteral, matchTypeIdentifier, matchMultilineComment, matchComment.* Estos métodos reciben una variable de tipo char y una variable de tipo *Token*. A partir del caracter inicial recibido, verifican si lo que le sigue en el archivo de entrada coincide con lo que se espera para un token en particular. Devuelven un booleano indicando si se pudo construir por completo el token esperado y actualizan el contenido del token recibido como parámetro. Pueden arrojar excepciones en algunos casos de token mal formado o caracteres inválidos.

matchIntLiteral: recibe una variable char y una variable de tipo *Token*. A partir del caracter inicial recibido, verifica si lo que le sigue es un literal numérico entero. Devuelve true si el token analizado es literal entero y actualiza los atributos del token recibido. Si el lexema estuviera seguido (sin espacios de por medio) por un token identificador, este método arroja un error de tipo *BadIdentifierError*.

matchStringLiteral: recibe una variable char y una variable de tipo *Token*. A partir del

caracter inicial, se encarga de detectar si el token actual es un literal de cadena. Devuelve true si ese fuera el caso y actualiza los atributos del token recibido. Si encuentra un *nil* o salto de línea, arroja un error de tipo *InvalidLiteralError*.

matchCharLiteral: análogo a *matchStringLiteral.* Este método se encarga de detectar si el token que está siendo analizado corresponde a un único caracter encerrado entre comillas simples. Si encuentra un *nil* o salto de línea, arroja un error de tipo *InvalidLiteralError*.

matchIdentifier: recibe una variable char y una variable de tipo *Token*. Su comportamiento es como el de un autómata finito determinista con 4 estados. Este se encarga de verificar que el identificador comience con un caracter del alfabeto español (exceptuando la ñ) o con un guión bajo. Si es así, continúa consumiendo caracteres siempre y cuando sean dígitos, guiones bajos o letras del abecedario. Si encuentra un espacio en blanco, un fin de archivo, o algún operador listado en el método *isOperator* frena la ejecución y acepta la cadena de entrada como un token de tipo identificador. En caso de tratarse de un identificador mal formado (en caso de incluir caracteres del alfabeto no válidos para un identificador), arroja el error de tipo *BadIdentifierError*.

matchTypeIdentifier: este método recibe un token de tipo identificador y verifica que el lexema del identificador encontrado comience con mayúscula. De ser así, el identificador pasa a ser identificador de tipo.

matchMultilineComment: recibe una variable char y una variable tipo *Token*. Actúa como un autómata finito determinista de 5 estados. Verifica que la cadena encontrada comience con “/*”. Si es así, consume cualquier caracter perteneciente al alfabeto de entrada hasta encontrar la secuencia de cierre, “*/”. Si se encuentra con el fin del archivo antes de encontrar el cierre del comentario multilínea, arroja un error de tipo *UnclosedMultilineCommentError*. Al ser un comentario, al identificarse este token no se agrega a la lista de tokens para el analizador sintáctico, sino que la función *nextToken* procede a buscar de forma recursiva el próximo token que no sea un comentario.

matchComment: recibe una variable char y una variable tipo *Token*. Actúa como un autómata finito determinista de 4 estados. Verifica si el token que se está analizando comienza con “//”. De ser así, consume cualquier caracter perteneciente al alfabeto de entrada hasta que se encuentra con un salto de línea o el final del archivo. Igual que con el comentario multilinea, al identificarse este token no se agrega a la lista de tokens para el analizador sintáctico, sino que la función *nextToken* procede a buscar de forma recursiva el próximo token que no sea un comentario.

Otros métodos:

consumeSpaces: se encarga de descartar espacios en blanco del archivo de entrada. Arroja *InvalidCharacterError* si se encuentra con algún caracter no perteneciente al

alfabeto de entrada

readConsumeChar: lee el siguiente caracter del archivo, lo consume y aumenta el contador de columna. Es decir, una vez leído no hay vuelta atrás. Si se encuentra con el fin de una línea, verifica si hay una línea más. De ser así, incrementa el contador de línea, reinicia el contador de columna y continúa leyendo la siguiente línea del archivo. Si encuentra el final del archivo, marca que se llegó al final de la lectura y cierra el archivo.

readWithoutConsumeChar: análogo a *readConsumeChar*, lee el siguiente caracter del archivo pero no lo consume. Es decir, no se pierde el caracter que consumido. Si no se invoca a la función *readConsumeChar*, la función *readWithoutConsumeChar* devuelve siempre el mismo caracter en llamados sucesivos.

Token

Esta clase representa cualquier token perteneciente a TinyRust+. Cada instancia guarda el número de línea y columna en donde fue encontrado dicho token junto a su lexema y tipo. Al ser todas sus variables privadas, esta clase cuenta con los getters y setters necesarios para accederlas y modificarlas. Esta clase extiende de la clase *Location*, que es usada en todo el proyecto para guardar ubicaciones del archivo de entrada. Mientras que la superclase *Location* guarda la línea y columna, la subclase *Token* guarda el lexema y el tipo.

ReservedWords

Esta clase estática contiene dos métodos estáticos: *matchReservedWord* que decide si un identificador es una palabra reservada y *matchPrimitiveType* que decide si un identificador de clase pertenece a una clase primitiva de TinyRust+.

El método *matchReservedWord* recibe una variable de tipo *Token* y compara su lexema contra la lista de palabras reservadas de TinyRust+. Si este presenta coincidencia con alguna palabra reservada, entonces se actualiza el tipo del token con el de la palabra reservada correspondiente.

El método *matchPrimitiveType* recibe una variable de tipo *Token* y compara su lexema contra la lista de nombres de clases predefinidas para tipos primitivos de TinyRust+. Si este presenta coincidencia con alguna clase predefinida, entonces se actualiza el tipo del token con el del tipo primitivo correspondiente.

Análisis sintáctico

El analizador sintáctico es un programa que consume los *tokens* generados por el analizador léxico y verifica que el programa cumpla con las reglas de la gramática de TinyRust+, es decir, que el archivo de entrada tenga una estructura sintácticamente válida para TinyRust+. Esta implementación consiste en un analizador sintáctico descendente predictivo recursivo.

Previo a su implementación, se tuvo que tratar la gramática de TinyRust+ en formato EBNF transformándola a notación BNF, eliminando la recursividad a izquierda y factorizando. Antes de explicar la implementación, se presentan las distintas gramáticas de TinyRust+. En una etapa posterior, esta gramática se enriqueció con acciones semánticas, su resultado se puede ver en la sección correspondiente al análisis semántico.

Gramática

En la especificación del lenguaje TinyRust+ se incluye una versión de su gramática en formato EBNF. Para hacer uso de la gramática para la implementación del analizador sintáctico descendente predictivo recursivo, se aplicaron una serie de algoritmos para transformar la gramática, sin alterar el conjunto de cadenas generados por la gramática. Los algoritmos aplicados fueron: transformación a formato BNF, eliminación de reglas lambda, eliminación de recursividad a izquierda y factorización. Finalmente, se calcularon el conjunto de primeros y siguientes para los símbolos no terminales de la gramática. En caso de modificarse la gramática del lenguaje, deben repetirse todos estos pasos para las reglas afectadas y modificarse la implementación del analizador sintáctico de forma acorde.

El lenguaje TinyRust+ es LL(1), esto significa que a partir de un símbolo no terminal de la gramática, consultando únicamente un token siguiente siempre es posible determinar unívocamente la regla de derivación que se debe aplicar. La gramática por sí sola es inherentemente ambigua, debido a la sintaxis del método *if*, donde el último bloque *else* de una seguidilla de condicionales puede ser ambiguamente asociado tanto a la primera como a la última sentencia *if*. Este tipo de ambiguedad y las de tipo aritmética se pueden salvar estableciendo un orden de precedencia en la semántica del lenguaje, lo que permite que este sea LL(1) a pesar de que su gramática es ambigua. Dado que es LL(1), es posible obtener una gramática sin recursividad a la izquierda y factorizada que genere el lenguaje y permita la implementación de un analizador sintáctico descendente, predictivo y recursivo.

BNF Extendida

⟨program⟩ ::= ⟨Clase⟩* ⟨Método-Main⟩

⟨Método-Main⟩ ::= **`fn main ()`** ⟨Bloque-Método⟩

⟨Clase⟩ ::= **`class id`** ⟨Herencia⟩**`?`** **`{`** ⟨Miembro⟩**`*`** **`}`**

⟨Herencia⟩ ::= : ⟨Tipo⟩

⟨Miembro⟩ ::= ⟨Atributo⟩ | ⟨Método⟩ | ⟨Constructor⟩

⟨Constructor ⟩ ::= **create** ⟨Argumentos-Formales⟩ ⟨Bloque-Método⟩

⟨Atributo⟩ ::= ⟨Visibilidad ⟩**?** ⟨Tipo⟩**:** ⟨Lista-Declaración-Variables⟩**;**

⟨Método⟩ ::= ⟨Forma-Método⟩**?** **fn id** ⟨Argumentos-Formales⟩ **->**
⟨Tipo-Método⟩ ⟨Bloque-Método⟩

⟨Visibilidad ⟩ ::= **pub**

⟨Forma-Método⟩ ::= **static**

⟨Bloque-Método⟩ ::= **{** ⟨Decl-Var-Locales⟩***** ⟨Sentencia⟩***** **}**

⟨Decl-Var-Locales⟩ ::= ⟨Tipo⟩**:** ⟨Lista-Declaración-Variables⟩**;**

⟨Lista-Declaración-Variables⟩::= **id** | **id** **,** ⟨Lista-Declaración-Variables⟩

⟨Argumentos-Formales⟩::= **(** ⟨Lista-Argumentos-Formales⟩**?** **)**

⟨Lista-Argumentos-Formales⟩ ::= ⟨Argumento-Formal ⟩**,**
⟨Lista-Argumentos-Formales⟩ | ⟨Argumento-Formal ⟩

⟨Argumento-Formal ⟩ ::= ⟨Tipo⟩**:** **id**

⟨TipoMétodo⟩ ::= ⟨Tipo⟩ | **void**

⟨Tipo⟩ ::= ⟨Tipo-Primitivo⟩ | ⟨Tipo-Referencia⟩ | ⟨Tipo-Arreglo⟩

⟨Tipo-Primitivo⟩ ::= **Str** | **Bool** |**I32** | **Char**

⟨Tipo-Referencia⟩ ::= **id**

⟨Tipo-Arreglo⟩ ::= **Array** ⟨Tipo-Primitivo⟩

⟨Sentencia⟩ ::= **;**
| ⟨Asignación⟩**;**
| ⟨Sentencia-Simple⟩**;**
| **if** **(**⟨Expresión⟩**)** ⟨Sentencia⟩
| **if** **(**⟨Expresión⟩**)** ⟨Sentencia⟩ **else** ⟨Sentencia⟩
| **while** **(** ⟨Expresión⟩ **)** ⟨Sentencia⟩
| ⟨Bloque⟩
| **return** ⟨Expresión⟩**?;**

⟨Bloque⟩ ::= **{** ⟨Sentencia⟩***** **}**

⟨Asignación⟩ ::= ⟨Acceso-Var-Simple⟩ **=** ⟨Expresión⟩
| ⟨Acceso-Self-Simple⟩ **=** ⟨Expresión⟩

⟨Acceso-Var-Simple⟩ ::= **id** ⟨Encadenado-Simple⟩* | **id [** ⟨Expresión⟩ **]**

⟨Acceso-Self-Simple⟩ ::= **self** ⟨Encadenado-Simple⟩*

⟨Encadenado-Simple⟩ ::= **.id**

⟨Sentencia-Simple⟩ ::= **(** ⟨Expresión⟩ **)**

⟨Expresión⟩ ::= ⟨ExpOr ⟩

⟨ExpOr ⟩ ::= ⟨ExpOr ⟩ **||** ⟨ExpAnd ⟩ | ⟨ExpAnd ⟩

⟨ExpAnd ⟩ ::= ⟨ExpAnd ⟩ **&&** ⟨ExpIgual ⟩ | ⟨ExpIgual ⟩

⟨ExpIgual ⟩ ::= ⟨ExpIgual ⟩⟨OpIgual ⟩⟨ExpCompuesta⟩ | ⟨ExpCompuesta⟩

⟨ExpCompuesta⟩ ::= ⟨ExpAd ⟩⟨OpCompuesto⟩⟨ExpAd ⟩ | ⟨ExpAd ⟩

⟨ExpAd ⟩ ::= ⟨ExpAd ⟩⟨OpAd ⟩⟨ExpMul ⟩ | ⟨ExpMul ⟩

⟨ExpMul ⟩ ::= ⟨ExpMul ⟩⟨OpMul ⟩⟨ExpUn⟩ | ⟨ExpUn⟩

⟨ExpUn⟩ ::= ⟨OpUnario⟩⟨ExpUn⟩ | ⟨Operando⟩

⟨OpIgual ⟩ ::= **==** | **!=**

⟨OpCompuesto⟩ ::= **<** | **>** | **<=** | **>=**

⟨OpAd ⟩ ::= **+** | **-**

⟨OpUnario⟩ ::= **+** | **-** | **!**

⟨OpMul ⟩ ::= ***** | **/** | **%**

⟨Operando⟩ ::= ⟨Literal ⟩ | ⟨Primario⟩⟨Encadenado⟩**?**

⟨Literal ⟩ ::= **nil** | **true** | **false** | **intLiteral** | **StrLiteral**
| **charLiteral**

⟨Primario⟩ ::= ⟨Expresión-Parentizada⟩
| ⟨Acceso-Self⟩
| ⟨Acceso-Var⟩
| ⟨Llamada-Método⟩
| ⟨Llamada-Método-Estático⟩
| ⟨Llamada-Constructor⟩

⟨Expresión-Parentizada⟩ ::= **(** ⟨Expresión⟩ **)** ⟨Encadenado⟩?

⟨Acceso-Self ⟩ ::= **self** ⟨Encadenado⟩?

⟨Acceso-Var ⟩ ::= **id** ⟨Encadenado⟩? | **id [** ⟨Expresión⟩ **]**

⟨Llamada-Método⟩ ::= **id** ⟨Argumentos-Actuales⟩ ⟨Encadenado⟩?

⟨Llamada-Método-Estático⟩ ::= **id .** ⟨Llamada-Método⟩ ⟨Encadenado⟩?

⟨Llamada-Constructor ⟩ ::= **new id** ⟨Argumentos-Actuales⟩ ⟨Encadenado⟩?
| **new** ⟨Tipo-Primitivo⟩ **[** ⟨Expresión⟩ **]**

⟨Argumentos-Actuales⟩ ::= **(** ⟨Lista-Expresiones⟩? **)**

⟨Lista-Expresiones⟩ ::= ⟨Expresión⟩ | ⟨Expresión⟩ **,** ⟨Lista-Expresiones⟩

⟨Encadenado⟩ ::= **.**⟨Llamada-Método-Encadenado⟩ | **.**⟨Acceso-Variable-Encadenado⟩

⟨Llamada-Método-Encadenado⟩ ::= **id** ⟨Argumentos-Actuales⟩ ⟨Encadenado⟩?

⟨Acceso-Variable-Encadenado⟩ ::= **id** ⟨Encadenado⟩? | **id [** ⟨Expresión⟩ **]**

Gramática final

Se encuentra a continuación la gramática final completa y ordenada obtenida tras aplicar todos los algoritmos mencionados. Esta gramática fue la que se utilizó posteriormente para implementar el analizador sintáctico descendente predictivo recursivo.

Start ::= Clases Main
Clases ::= Clase Clases | λ

Main ::= "fn" "main" "(" ")" Bloque-Método
Miembros ::= Miembro Miembros | λ

Clase ::= "class" "idClase" ClaseHerenciaOp
ClaseHerenciaOp ::= Herencia "{" Miembros "}" | "{" Miembros "}"

Herencia ::= ":" "idClase"
Miembro ::= Atributo | Constructor | Método
Atributo ::= Tipo ":" Lista-Declaración-Variables ";"
| Visibilidad Tipo ":" Lista-Declaración-Variables ";"

Constructor ::= "create" Argumentos-Formales Bloque-Método
Método ::= "fn" "idMétodoVariable" Argumentos-Formales "->" Tipo-Método Bloque-Método
| Forma-Método "fn" "idMétodoVariable" Argumentos-Formales "->" Tipo-Método Bloque-Método

```
Argumentos-Formales ::= "(" ")"    | "(" Lista-Argumentos-Formales ")"

Lista-Argumentos-Formales ::= Argumento-Formal "," Lista-Argumentos-Formales |
Argumento-Formal
Argumento-Formal ::= Tipo ":" "idMétodoVariable"
Forma-Método ::= "static"
Visibilidad ::= "pub"
Tipo-Método ::= Tipo | "void"
Tipo ::= Tipo-Primitivo | Tipo-Referencia | Tipo-Array
Tipo-Primitivo ::= "Bool" | "I32" | "Str" | "Char"
Tipo-Referencia ::= "idClase"
Tipo-Array ::= "Array" Tipo-Primitivo
Lista-Declaración-Variables ::= "idMétodoVariable" | "idMétodoVariable" ","
Lista-Declaración-Variables
Bloque-Método ::= "{" Decl-Var-LocalesN Sentencias "}"

Decl-Var-LocalesN := Decl-Var-Locales Decl-Var-LocalesN | λ

Decl-Var-Locales ::= Tipo ":" Lista-Declaración-Variables ";"

Sentencias ::= Sentencia Sentencias | λ

Sentencia ::= ";"
      | Asignación ";"
      | Sentencia-Simple ";"
      | "if" "(" Expresión ")" Sentencia ElseOp
      | "while" "(" Expresión ")" Sentencia
      | Bloque
      | "return" Expresión ";"
      | "return" ";"

ElseOp ::= "else" Sentencia | λ

Bloque ::= "{" Sentencias "}"

Asignación ::= Asignación-Variable-Simple "=" Expresión | Asignación-Self-Simple
"=" Expresión
Asignación-Variable-Simple  ::= "idMétodoVariable" "[" Expresión "]" |
"idMétodoVariable" Encadenado-SimpleN

Encadenado-SimpleN ::= Encadenado-Simple Encadenado-SimpleN | λ

Asignación-Self-Simple ::= "self" Encadenado-SimpleN

Encadenado-Simple ::= "." "idMétodoVariable"

Sentencia-Simple ::= "(" Expresión ")"

Lista-Expresiones ::= Expresión | Expresión "," Lista-Expresiones

Expresión ::= ExpOr
```

```
ExpOr ::= ExpAnd ExpOrP
ExpOrP ::= "||" ExpAnd ExpOrP | λ
ExpAnd ::= ExpIgual ExpAndP
ExpAndP ::= "&&" ExpIgual ExpAndP | λ
ExpIgual ::= ExpCompuesta ExpIgualP
ExpIgualP ::= OpIgual ExpCompuesta ExpIgualP | λ
ExpCompuesta ::= ExpAdd OpCompuesto ExpAdd | ExpAdd
ExpAdd ::= ExpMul ExpAddP
ExpAddP ::= OpAdd ExpMul ExpAddP | λ
ExpMul ::= ExpUn ExpMulP
ExpMulP ::= OpMul ExpUn ExpMulP | λ
ExpUn ::= OpUnario ExpUn | Operando

OpIgual ::= "==" | "!="
OpCompuesto ::= "<" | ">" | "<=" | ">="
OpAdd ::= "+" | "-"
OpUnario ::= "+" | "-" | "!"
OpMul ::= "*" | "/" | "%"
Operando ::= Literal | Primario EncadenadoOp
Literal ::= "nil" | "true" | "false" | "intLiteral" | "stringLiteral" | "charLiteral"
Primario ::= ExpresionParentizada | AccesoSelf | AccesoVar  | Llamada-Método  |
Llamada-Método-Estático | Llamada-Constructor
ExpresionParentizada ::= "(" Expresión ")" EncadenadoOp
AccesoSelf ::= "self" EncadenadoOp
AccesoVar ::= "idMétodoVariable" EncadenadoOp | "idMétodoVariable"  "["
Expresión "]"
Llamada-Método ::= "idMétodoVariable" Argumentos-Actuales EncadenadoOp
Llamada-Método-Estático ::= "idClase" "." Llamada-Método EncadenadoOp
Llamada-Constructor ::= "new" "idClase" Argumentos-Actuales EncadenadoOp
    | "new" Tipo-Primitivo "[" Expresión "]"

Argumentos-Actuales ::= "(" ")" | "(" Lista-Expresiones ")"
Encadenado ::= "." Llamada-Método-Encadenado | "." Acceso-Variable-Encadenado

Llamada-Método-Encadenado ::= "idMétodoVariable" Argumentos-Actuales
EncadenadoOp
Acceso-Variable-Encadenado ::= "idMétodoVariable" EncadenadoOp |
"idMétodoVariable" "[" Expresión "]"
EncadenadoOp ::= Encadenado | λ
```

Primeros

A partir de la versión final de la gramática, se procedió a calcular la lista de primeros y siguientes para cada símbolo no terminal de la gramática. La lista obtenida es la siguiente:

Start = {"class", "fn"}
Clases = {"class", λ}
Clase = {"class"}
Main = {"fn"}

Miembros = {"Bool", "I32", "Str", "Char", "idClase", "Array", "pub", "create", "fn", "static", λ}
Herencia = {":"}
ClaseHerenciaOp = {":", "{"}
Miembro = {"Bool", "I32", "Str", "Char", "idClase", "Array", "pub", "create", "fn", "static"}
Atributo = {"Bool", "I32", "Str", "Char", "idClase", "Array", "pub"}

Constructor = {"create"}
Método = {"fn", "static"}
Argumentos-Formales = {"("}
Lista-Argumentos-Formales = {"Bool", "I32", "Str", "Char", "idClase", "Array"}
Argumento-Formal = {"Bool", "I32", "Str", "Char", "idClase", "Array"}
Forma-Método = {"static"}
Visibilidad = {"pub"}
Tipo-Método = {"Bool", "I32", "Str", "Char", "idClase", "Array", "void"}
Tipo = {"Bool", "I32", "Str", "Char", "idClase", "Array"}
Tipo-Primitivo = {"Bool", "I32", "Str", "Char"}
Tipo-Referencia = {"idClase"}
Tipo-Array = {"Array"}
Lista-Declaración-Variables = {"idMétodoVariable"}
Bloque-Método = {"{"}

Decl-Var-LocalesN = {"Bool", "I32", "Str", "Char", "idClase", "Array", λ }
Decl-Var-Locales = {"Bool", "I32", "Str", "Char", "idClase", "Array"}

Sentencias = {";", "idMétodoVariable", "self", "(", "if", "while", "{", "return", λ}
Sentencia = {";", "idMétodoVariable", "self", "(", "if", "while", "{", "return"}
ElseOp = {"else", λ}
Bloque = {"{"}

Asignación = {"idMétodoVariable", "self"}
Asignación-Variable-Simple = {"idMétodoVariable"}

Encadenado-SimpleN = {".", λ}
Asignación-Self-Simple = {"self"}
Encadenado-Simple = {"."}
Sentencia-Simple = {"("}

Lista-Expresiones = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}

Expresión = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}

ExpOr = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}
ExpOrP = {"||", λ}

ExpAnd = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}
ExpAndP = {"&&", λ}

ExpIgual = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}
ExpIgualP = {"==", "!=", λ}

ExpCompuesta = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}

ExpAdd = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}
ExpAddP = {"+", "-", λ}

ExpMul = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}
ExpMulP = {"*", "/", "%", λ}

ExpUn = {"+", "-", "!", "nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new"}

OpIgual = {"==", "!="}
OpCompuesto = {"<", ">", "<=", ">="}
OpAdd = {"+", "-"}
OpUnario = {"+", "-", "!"}
OpMul = {"*", "/", "%"}
Operando = {"nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral", "(", "self", "idMétodoVariable" , "idClase", "new" }
Literal = {"nil", "true", "false", "intLiteral", "stringLiteral", "charLiteral"}
Primario = {"(", "self", "idMétodoVariable" , "idClase", "new"}
ExpresionParentizada = {"("}
AccesoSelf = {"self"}
AccesoVar = {"idMétodoVariable"}
Llamada-Método = {"idMétodoVariable"}
Llamada-Método-Estático = {"idClase"}
Llamada-Constructor = {"new"}

Argumentos-Actuales = {"("}

Encadenado = {"."}
Llamada-Método-Encadenado = {"idMétodoVariable"}
Acceso-Variable-Encadenado = {"idMétodoVariable"}
EncadenadoOp = {".", λ}

Siguientes

El cálculo de siguientes se realizó únicamente para los símbolos cuyos primeros incluyen lambda. Esta decisión es con fines de practicidad, ya que durante la implementación del analizador sintáctico de un lenguaje LL1, se requerirá a lo sumo saber los siguientes de los símbolos que derivan lambda. Posteriormente la implementación se pudo lograr sin hacer uso en lo absoluto de este conjunto de siguientes, pero el cálculo se realizó de todos modos y el conjunto de siguientes para las reglas que derivan lambda se encuentra a continuación.

Clases = {"fn"}
Miembros = {"}"}
Sentencias = {"}"}
ElseOp = {"else", ";", "idMétodoVariable", "self", "(", "if", "while", "{", "return", "}"}
Decl-Var-LocalesN = {";", "idMétodoVariable", "self", "(", "if", "while", "{", "return", "}"}
Encadenado-SimpleN = {"="}
ExpOrP = {";", ")", "]", ","}
ExpAndP = {"||", ";", ")", "]", ","}
ExpIgualP = {"&&", "||", ";", ")", "]", ","}
ExpAddP = {"<", ">", "<=", ">=", "==", "!=", "&&", "||", ";", ")", "]", ","}
ExpMulP = { "+", "-", "<", ">", "<=", ">=", "==", "!=", "&&", "||", ";", ")", "]", ","}
EncadenadoOp = {".", "*", "/", "%", "+", "-", "<", ">", "<=", ">=", "==", "!=", "&&", "||", ";", ")", "]", ","}

Errores

Para la etapa de análisis sintáctico se previó un único tipo de error *Token inesperado*, generado cuando se encuentra un token que no coincide con ninguna regla gramatical de TinyRust+, teniendo en cuenta la secuencia previa de tokens del archivo.

Acorde a esto, todos los errores lanzados en esta etapa son de tipo *UnexpectedToken*, que hereda de la clase *SyntacticalError*. Esta excepción se produce cuando se encuentra un token que no coincide con ninguna regla gramatical de TinyRust+, teniendo en cuenta la secuencia previa de tokens. Este error incluye en cada caso un mensaje especificando el lexema que se encontró y el o los posibles tokens/lexemas que se esperaba encontrar en ese contexto. En ocasiones este error incluye una descripción de la estructura gramatical que se esperaba en vez de incluir la especificación de cada posible token esperado. Por ejemplo, cuando se espera una expresión y se recibe un token que no podría ser el primer token de ninguna expresión válida, el mensaje dirá “se esperaba una expresión, se encontró [token]”. Esto es debido a que un mensaje listando el conjunto de 14 tokens

válidos para comenzar una expresión podría no resultar de ayuda para el usuario que intenta compilar un archivo de TinyRust+.

Implementación

El punto de entrada del analizador sintáctico es la clase *Executor*. La implementación de toda la funcionalidad de esta etapa se encuentra dentro de la clase *SyntacticAnalyzer*.

Executor

Esta clase consta de un único método llamado *run*. Al invocar este método, se crea una instancia de la clase *SyntacticAnalyzer* que recibe como parámetro la ruta del archivo de entrada. Una vez instanciado el analizador, el ejecutor llamará al método *run* del *SyntacticAnalyzer*, el cual devuelve un booleano indicando si el archivo de entrada es un archivo sintácticamente válido de TinyRust+. En caso de un resultado positivo, un mensaje de éxito es escrito en el archivo de salida o mostrado por pantalla. Si el archivo no fuera sintácticamente correcto, esta clase intercepta la excepción lanzada por *SyntacticAnalyzer* y en la salida muestra fila, columna y token donde se presentó el error, incluyendo en la descripción si se trata de un error léxico o sintáctico.

SyntacticAnalyzer

Esta clase es la encargada de todo el proceso de análisis sintáctico. El análisis se hace de forma descendente y predictiva, mientras que la implementación en esta clase es recursiva.

En su constructor se inicializa una instancia de *LexicAnalyzer* pasándole la ruta del archivo de entrada.

Inicialmente esta clase sólo se encargaba del análisis semántico, pero tras el avance del proyecto, se implementó dentro de esta clase la construcción de la tabla de símbolos y del árbol sintáctico abstracto, dos estructuras que son utilizadas para el análisis semántico. En el constructor de SyntacticAnalyzer se inicializan estas dos estructuras: una instancia de *SymbolTable* y una instancia de *AbstractSyntaxTree*. Si bien su construcción está inevitablemente ligada al análisis sintáctico, se puede elegir obviar la etapa de validación de ambas estructuras para enfocarse en el funcionamiento del analizador sintáctico.

SyntacticAnalyzer provee el método público *run*. Este método comienza el proceso de análisis sintáctico del archivo utilizando la gramática de TinyRust+. Durante el análisis sintáctico, se irán solicitando y consumiendo tokens del analizador léxico y se verificará que el archivo de entrada sea derivable a partir del símbolo inicial de la gramática, llamando recursivamente métodos de la clase que implementan cada regla de la gramática.

La clase tiene dos atributos privados relevantes: *token* y *nextToken*. Ambos son instancias de la clase Token y son generados por el analizador léxico. Ambas variables son inicializadas con los primeros dos tokens del archivo al ejecutar el método *run.*

Luego su valor va cambiando a medida que el análisis sintáctico avanza y los tokens son *matcheados* en reglas de la gramática. Cada vez que se *matchea* y consume un token, se mueve el valor de *nextToken* a *token* y se solicita un nuevo token al analizador léxico para actualizar el nextToken. En caso de que el analizador léxico arroje una excepción, el analizador sintáctico no la atrapa, sino que la propaga hacia arriba al siguiente invocador.

A su vez, *SyntacticAnalyzer* posee una enorme cantidad de métodos privados, algunos de ellos se describen a continuación:

matchToken: compara el tipo del token actual contra la lista de tipos de token recibida. Si coincide con algún elemento de la lista, consume el token. Caso contrario, lanza una excepción de tipo *UnexpectedToken*.

matchLexema: compara el lexema del token actual contra la lista de lexemas recibida. Si coincide con algún elemento de la lista, consume el token. Caso contrario, lanza una excepción de tipo *UnexpectedToken*.

isFirstT: compara el tipo del token actual contra la lista de tipos de token recibida y retorna un booleano indicando si coincide con algún elemento de la lista. La principal diferencia con *matchToken* es que *isFirstT* lee el token sin consumirlo.

isFirstL: compara el lexema del token actual contra la lista de lexemas recibida y retorna un booleano indicando si coincide con algún elemento de la lista. La principal diferencia con *matchLexema* es que *isFirstL* lee el token sin consumirlo.

El resto de métodos privados de la clase *SyntacticAnalyzer* se corresponden cada uno con una regla (homónima) de la gramática final de TinyRust+. Cada método implementa no solo la lógica de *matcheo* sintáctico de la regla correspondiente, sino también las acciones semánticas de la regla. El método correspondiente a una regla se ejecutará por completo sin problemas si la secuencia de tokens encontrados es derivable mediante esa regla de la gramática y no infringe ninguna regla semántica al agregar entidades a la TS. Mientras *matchea* tokens, la regla puede agregar entidades a la tabla de símbolos, modificar el estado de la tabla de símbolos o construir y componer nodos del árbol sintáctico abstracto. Cuando el método termina de ejecutar, posiblemente retorne un nodo del AST o una entidad para la TS, dependiendo de la regla. En caso de que la entrada no sea correcta sintácticamente o se agregue una entidad inválida a la TS, la regla arrojará una excepción y no completará su ejecución.

Análisis semántico

En el análisis semántico se verifica que el archivo de entrada respete las restricciones de tipos, entidades, identificadores y demás reglas semánticas del lenguaje TinyRust+. Para realizar este análisis, durante el análisis sintáctico se construyen dos estructuras: la tabla de símbolos y el árbol sintáctico abstracto). Cuando el archivo de entrada se ha consumido exitosamente y estas dos estructuras se han construído por completo, serán consolidadas y validadas en una segunda pasada para garantizar la validez semántica de la entrada.

La tabla de símbolos (TS, por sus siglas abreviadas) se construye de forma intercalada a medida que se realiza el análisis sintáctico del archivo de entrada. Durante el análisis sintáctico se agregan entidades a la tabla y para cuando se ha completado el análisis exitosamente, la tabla de símbolos está completa y se realiza su consolidación.

La TS es una estructura que guarda registro de todas las entidades (clases, métodos, atributos, argumentos y variables) declarados dentro del programa de TinyRust+ y tiene forma arbórea. Se inicializa como una estructura vacía antes de comenzar el análisis sintáctico y durante el análisis sintáctico se construye desde la raíz hasta sus hojas. Cuando una regla sintáctica hace match, se utiliza la información de los tokens consumidos en esa regla para generar una entrada nueva de la TS.

El árbol sintáctico abstracto (AST, por sus siglas en inglés) se construye de forma recursiva en simultáneo con el análisis sintáctico y la construcción de la tabla de símbolos. Para cuando se ha completado la construcción del AST, se realizan sobre él las validaciones de sentencias del archivo.

El AST se inicializa como una estructura vacía antes de comenzar el análisis sintáctico y durante el análisis sintáctico se construye desde sus hojas hasta la raíz. Luego de que una regla sintáctica haga match, se utiliza la información de los tokens consumidos en esa regla para generar un nodo del AST y se retorna en la regla. Cuando esta regla es *matcheada* en una regla superior, se utiliza el nodo obtenido para construir a su vez el nodo que retornará la regla superior, formando así un árbol de nodos.

De este modo, los primeros nodos que se generan son los de literales y variables, que luego se anexan a los nodos de las expresiones. Los nodos de expresiones son utilizados al armar los nodos de las sentencias; los nodos de sentencias se anexan a los nodos bloque y luego los nodos de bloques se anexan al nodo del método correspondiente. Finalmente, los nodos de método se guardan dentro de los nodos de clase y el nodo raíz del AST contiene únicamente una lista de clases.

Luego de construir el AST por completo y consolidar la tabla de símbolos, se procede a realizar la validación del AST, que se realiza también de forma recursiva, pero esta vez se realiza desde la raíz hasta las hojas del árbol.

La validación del nodo raíz del AST consiste en validar cada nodo clase, mientras que la validación de un nodo clase consiste en validar todos sus métodos. Luego la validación de

cada método es más diversa, se utiliza la tabla de símbolos para corroborar que las entidades utilizadas en cada sentencia del método existen, que existan las propiedades accedidas en cada entidad y que se utilicen entidades del tipo correcto en cada contexto (como asignaciones, operaciones, retornos, etc).

Esquema de traducción

Para la implementación del analizador semántico de declaraciones y sentencias, la gramática tratada para el análisis sintáctico se extendió con atributos y acciones semánticas para la construcción de la TS y el AST. A continuación se encuentra el esquema de traducción. Estas acciones semánticas posteriormente se implementaron dentro del código del analizador sintáctico dentro de cada regla correspondiente.

Como referencia, las acciones semánticas de color verde o azul están relacionadas con la TS, mientras que las acciones anaranjadas o rojas se relacionan al AST. Las acciones de color verde crean o agregan entidades a la tabla de símbolos, mientras que las anotaciones azules unicamente sintetizan o establecen atributos que serán utilizados para la construcción de la TS. Las acciones rojas crean nodos hoja del AST, mientras que las anotaciones anaranjadas sintetizan nodos o modifican el AST.

```
Start ::= Clases Main {ast.setClases(Clases.nodo)}
Clases(1) ::= Clase Clases(2) {Clases1.nodo = Clases2.nodo.add(Clase.nodo)}
    | λ  {Clases1.nodo = new TreeList<Clase>()}

Main ::=  "fn" "main" "(" ")" Bloque-Método {ts.addMain();
ast.addMain(BloqueMetodo.nodo);}

Clase ::= "class" "idClase" ClaseHerenciaOp {currClase = ts.addClase(idClase.lex);
Clase.nodo = new Clase(nombre: idClase.lex, metodos: CHO.nodo);}

Herencia ::= ":" "idClase" {currClase.heredaDe = idClase.lex;}

ClaseHerenciaOp ::= Herencia "{" Miembros "}" {CHO.nodo = Miembros.nodo}
    | "{" Miembros "}" {CHO.nodo = Miembros.nodo}

Miembros(1) ::= Miembro Miembros(2) {Miembros1.nodo =
Miembros2.nodo.addIfNotNull(Miembro.nodo)}
    | λ {Miembros1.nodo = new TreeList<Metodo>()}

Miembro ::= Atributo {Miembro.nodo = null}
    | Constructor {Miembro.nodo = Constructor.nodo}
    | Método {Miembro.nodo = Metodo.nodo}

Atributo ::= Tipo ":" Lista-Declaración-Variables ";" {LDV.tipo = Tipo.tipo}
    | Visibilidad Tipo ":" Lista-Declaración-Variables ";" {LDV.tipo = Tipo.tipo; LDV.public =
Visibilidad.public}

Constructor ::= "create" Argumentos-Formales Bloque-Método {currMetodo =
ts.addConstructorClase(currClase); Constructor.nodo = new Metodo(nombre: “create”,
bloque: BloqueMetodo.nodo);}
Método ::= "fn" "idMétodoVariable" Argumentos-Formales "->" Tipo-Método Bloque-Método
```

{currMetodo = ts.addMetodoClase(currClase, new Metodo(nombre: idMetodoVariable.lex, retornoTipo: TipoMetodo.tipo); Metodo.nodo = new Metodo(nombre: idMetodoVariable.lex, bloque: BloqueMetodo.nodo);}
| Forma-Método "fn" "idMétodoVariable" Argumentos-Formales "->" Tipo-Método Bloque-Método {currMetodo = ts.addMetodoClase(currClase, new Metodo(nombre: idMetodoVariable.lex, retornoTipo: TipoMetodo.tipo, static: FormaMetodo.static); Metodo.nodo = new Metodo(nombre: idMetodoVariable.lex, bloque: BloqueMetodo.nodo);}

Argumentos-Formales ::= "(" ")"
| "(" Lista-Argumentos-Formales ")"
Lista-Argumentos-Formales ::= Argumento-Formal "," Lista-Argumentos-Formales
| Argumento-Formal

Argumento-Formal ::= Tipo ":" "idMétodoVariable" {currMetodo.addArgumento(nombre: idMetodoVariable.lex, tipo: Tipo.tipo)}

Forma-Método ::= "static" {FormaMetodo.static = true}

Visibilidad ::= "pub" {Visibilidad.public = true}

Tipo-Método ::= Tipo {TipoMetodo.tipo = Tipo.tipo}
| "void" {TipoMetodo.tipo = new Void()}

Tipo ::= Tipo-Primitivo {Tipo.tipo = TipoPrimitivo.tipo}
| Tipo-Referencia {Tipo.tipo = TipoReferencia.tipo}
| Tipo-Array {Tipo.tipo = TipoArray.tipo}

Tipo-Primitivo ::= "Bool" {TipoPrimitivo.tipo = new Bool()}
| "I32" {TipoPrimitivo.tipo = new I32()}
| "Str" {TipoPrimitivo.tipo = new Str()}
| "Char" {TipoPrimitivo.tipo = new Char()}

Tipo-Referencia ::= "idClase" {TipoReferencia.tipo = new TipoClase(nombre:idClase.lex)}

Tipo-Array ::= "Array" Tipo-Primitivo {TipoArray.tipo = new Array(tipo: TipoPrimitivo.tipo)}

Lista-Declaración-Variables(1) ::= "idMétodoVariable" {ts.addVariable(new Var(name: idMetodoVariable.lex, tipo: LDV1.tipo, public: LDV1.public))}
| "idMétodoVariable" "," Lista-Declaración-Variables(2) {ts.addVariable(new Var(name: idMetodoVariable.lex, tipo: LDV1.tipo, public: LDV1.public)); LDV2.tipo = LDV1.tipo; LDV2.public = LDV1.public;}

Bloque-Método ::= "{" Decl-Var-LocalesN Sentencias "}" {BM.nodo = new Bloque(Sentencias.nodo)}

Decl-Var-LocalesN := Decl-Var-Locales Decl-Var-LocalesN | λ

Decl-Var-Locales ::= Tipo ":" Lista-Declaración-Variables ";" {LDV.tipo = Tipo.tipo}

Sentencias(1) ::= Sentencia Sentencias(2) {Sentencias(1).nodo =

```
Sentencias2.nodo.add(Sentencia.nodo)}
   | λ  {Sentencias(1).nodo = new TreeList<Sentencias>()}

Sentencia(1) ::= ";"  {Sentencia1.nodo = null}
      | Asignación ";"  {Sentencia1.nodo = Asignacion.nodo}
      | Sentencia-Simple ";"  {Sentencia1.nodo = SS.nodo}
      | "if" "(" Expresión ")" Sentencia(2) ElseOp {Sentencia1.nodo = new IfElse(cond:
Exp.nodo, bloque: Sentencia2.nodo, elseBloque: ElseOp.nodo)}
      | "while" "(" Expresión ")" Sentencia(2) {Sentencia1.nodo = new While(cond:
Expresion.nodo, bloque: Sentencia2.nodo)}
      | Bloque {Sentencia1.nodo = Bloque.nodo}
      | "return" Expresión ";" {Sentencia1.nodo = new Return(valor: Expresion.nodo)}
      | "return" ";" {Sentencia1.nodo = new Return(valor: new Void())}

ElseOp ::= "else" Sentencia {ElseOp.nodo = Sentencia.nodo}
   | λ {ElseOp.nodo = null}

Bloque ::= "{" Sentencias "}" {Bloque.nodo = new Bloque(Sentencias.nodo)}

Asignación ::= Asignación-Variable-Simple "=" Expresión {Asignacion.nodo = new
Asignacion(izq: AVS.nodo, der: Exp.nodo)}
   | Asignación-Self-Simple "=" Expresión {Asignacion.nodo = new Asignacion(izq:
ASS.nodo, der: Exp.nodo)}

Asignación-Variable-Simple  ::= "idMétodoVariable" "[" Expresión "]" {AVS.nodo = new
AccesoArray(var: new Var(idMetodoVar.lex), indice: Expresion.nodo)}
   | "idMétodoVariable" Encadenado-SimpleN {AVS.nodo = new AccesoVar(var: new
Var(idMetodoVar.lex), cadena: ESN.nodo)}

Encadenado-SimpleN(1) ::= Encadenado-Simple Encadenado-SimpleN(2) {ESN1.nodo = new
AccesoAttrEncadenado(var: ES.nodo, cadena: ESN2.nodo)}
   | λ {ESN1.nodo = null}

Asignación-Self-Simple ::= "self" Encadenado-SimpleN {ASS.nodo = new AccesoVar(var:
new Var("self"), cadena: ESN.nodo)}

Encadenado-Simple ::= "." "idMétodoVariable" {ES.nodo = new Var(idMetodoVar.lex)}

Sentencia-Simple ::= "(" Expresión ")" {SS.nodo = new SentSimple(Exp.nodo)}

Expresión ::= ExpOr {Expresion.nodo = ExpOr.nodo}

ExpOr ::= ExpAnd ExpOrP {if (ExpOrP.nodo != null) {
      ExpOr.nodo = ExpOrP.nodo.setIzq(ExpAnd.nodo);
      } else {ExpOr.nodo = ExpAnd.nodo;}}

ExpOrP(1) ::= "||" ExpAnd ExpOrP(2) {if (ExpOrP2.nodo != null) {
      ExpOrP2.setIzq(ExpAnd.nodo);
      ExpOrP1.nodo = new OpBinaria(op: "||", derecha: ExpOrP2.nodo);
      } else {ExpOrP1.nodo = new OpBinaria(op: "||", derecha: ExpAnd.nodo);}}
   | λ {ExpOrP.nodo = null}
```

```
ExpAnd ::= ExpIgual ExpAndP {if (ExpAndP.nodo != null) {
      ExpAnd.nodo = ExpAndP.nodo.setIzq(ExpIgual.nodo);
      } else {ExpAnd.nodo = ExpIgual.nodo;}}

ExpAndP(1) ::= "&&" ExpIgual ExpAndP(2) {if (ExpAndP2.nodo != null) {
      ExpAndP2.setIzq(ExpIgual.nodo);
      ExpAndP1.nodo = new OpBinaria(op: “&&”, derecha: ExpAndP2.nodo);
      } else {ExpAndP1.nodo = new OpBinaria(op: “&&”, derecha: ExpIgual.nodo);}}
   | λ {ExpIgualP.nodo = null}

ExpIgual ::= ExpCompuesta ExpIgualP  {if (ExpIgualP.nodo != null) {
      ExpIgual.nodo = ExpIgualP.nodo.setIzq(ExpCompuesta.nodo);
      } else {ExpIgual.nodo = ExpCompuesta.nodo;}}


ExpIgualP(1) ::= OpIgual ExpCompuesta ExpIgualP(2) {if (ExpIgualP2.nodo != null) {
      ExpIgualP2.setIzq(ExpComp.nodo);
      ExpIgualP1.nodo = new OpBinaria(op: OpIgual.lex, derecha: ExpIgualP2.nodo);
      } else {ExpIgualP1.nodo = new OpBinaria(op: OpIgual.lex, der: ExpComp.nodo);}}
   | λ {ExpIgualP.nodo = null}

ExpCompuesta ::= ExpAdd(1) OpCompuesto ExpAdd(2) {ExpComp.nodo = new
OpBinaria(op: OpComp.lex, izq: ExpAdd1.nodo, der: ExpAdd2.nodo)}
   | ExpAdd {ExpComp.nodo = ExpAdd.nodo}

ExpAdd ::= ExpMul ExpAddP  {if (ExpAddP.nodo != null) {
      ExpAdd.nodo = ExpAddP.nodo.setIzq(ExpMul.nodo);
      } else {ExpAdd.nodo = ExpMul.nodo;}}

ExpAddP(1) ::= OpAdd ExpMul ExpAddP(2)  {if (ExpAddP2.nodo != null) {
      ExpAddP2.setIzq(ExpMul.nodo);
      ExpAddP1.nodo = new OpBinaria(op: OpAdd.lex, derecha: ExpAddP2.nodo);
      } else {ExpAddP1.nodo = new OpBinaria(op: OpAdd.lex, der: ExpMul.nodo);}}
   | λ {ExpAddP.nodo = null}

ExpMul ::= ExpUn ExpMulP  {if (ExpMulP.nodo != null) {
ExpMul.nodo = ExpMulP.nodo.setIzq(ExpUn.nodo); else {ExpMul.nodo = ExpUn.nodo;}}

ExpMulP(1) ::= OpMul ExpUn ExpMulP(2) {if (ExpMulP2.nodo != null) {
      ExpMulP2.setIzq(ExpUn.nodo);
      ExpMulP1.nodo = new OpBinaria(op: OpMul.lex, derecha: ExpMulP2.nodo);
      } else {ExpMulP1.nodo = new OpBinaria(op: OpMul.lex, der: ExpUn.nodo);}}
   | λ {ExpMulP.nodo = null}

ExpUn(1) ::= OpUnario ExpUn(2) {ExpUn1.nodo = new ExpUnaria(op: OpUnario.lex, exp:
ExpUn2.nodo}
   | Operando {ExpUn1.nodo = Operando.nodo}

OpIgual ::= "=="
   | "!="
```

```
OpCompuesto ::= "<"
    | ">"
    | "<="
    | ">="

OpAdd ::= "+"
    | "-"

OpUnario ::= "+"
    | "-"
    | "!"
OpMul ::= "*"
    | "/"
    | "%"

Operando ::= Literal {Op.nodo = new Literal(Literal.lex)}
    | Primario EncadenadoOp {Op.nodo = new ExpEncadenada(exp: Primario.nodo, cadena:
EncadenadoOp.nodo)}}

Literal ::= "nil"
    | "true"
    | "false"
    | "intLiteral"
    | "stringLiteral"
    | "charLiteral"

Primario ::= ExpresionParentizada  {Primario.nodo = ExpresionParentizada.nodo}
    | AccesoSelf {Primario.nodo = AccesoSelf.nodo}
    | AccesoVar {Primario.nodo = AccesoVar.nodo}
    | Llamada-Método  {Primario.nodo = LlamadaMetodo.nodo}
    | Llamada-Método-Estático {Primario.nodo = LlamadaMetodoEstatico.nodo}
    | Llamada-Constructor {Primario.nodo = LlamadaConstructor.nodo}

ExpresionParentizada ::= "(" Expresión ")" EncadenadoOp {ExpParent.nodo = new
ExpEncadenada(exp: Expresion.nodo, cadena: EncadenadoOp.nodo}

Lista-Expresiones(1) ::= Expresión {LE1.nodo = new TreeList<Expresion>()}
    | Expresión "," Lista-Expresiones(2) {LE1.nodo = LE2.nodo.add(Exp.nodo)}

AccesoSelf ::= "self" EncadenadoOp {AccesoSelf.nodo = new AccesoVar(var: new
Var("self"), cadena: EncadenadoOp.nodo)}

AccesoVar ::= "idMétodoVariable" EncadenadoOp {AccesoVar.nodo = new AccesoVar(var:
new Var(idMetodoVariable.lex), cadena: EncadenadoOp.nodo)}
    | "idMétodoVariable"  "[" Expresión "]" {AccesoVar.nodo = new AccesoArray(var: new
Var(idMetodoVariable.lex), indice: Expresion.nodo)}

Llamada-Método ::= "idMétodoVariable" Argumentos-Actuales EncadenadoOp {LM.nodo =
new LlamadaMetodo(nombre: new Var(idMetodoVariable.lex), args:
ArgumentosActuales.nodo, acceso: EncadenadoOp.nodo)}
```

Llamada-Método-Estático ::= "idClase" "." Llamada-Método EncadenadoOp {LM.nodo = new LlamadaMetodoEstatico(clase: idClase.lex, llamado: LlamadaMetodo.nodo, acceso: EncadenadoOp.nodo)}

Llamada-Constructor ::= "new" "idClase" Argumentos-Actuales EncadenadoOp {LM.nodo = new LlamadaConstructor(args: ArgumentosActuales.nodo, acceso: EncadenadoOp.nodo)}
| "new" Tipo-Primitivo "[" Expresión "]" {LM.nodo = new LlamadaConstructorArray(tipo: TipoPrimitivo.nodo, tamaño: Expresion.nodo)}

Argumentos-Actuales ::= "(" ")" {ArgAct.nodo = new TreeList<Expresion>()}
| "(" Lista-Expresiones ")" {ArgAct.nodo = ListaExp.nodo}

Encadenado ::= "." Llamada-Método-Encadenado {Encadenado.nodo = LME.nodo}
| "." Acceso-Variable-Encadenado {Encadenado.nodo = AVE.nodo}

Llamada-Método-Encadenado ::= "idMétodoVariable" Argumentos-Actuales EncadenadoOp {LME.nodo = new LlamadaMetodoCadena(nombre: new Var(idMetodoVariable.lex), args: ArgumentosActuales.nodo, cadena: EncadenadoOp.nodo)}

Acceso-Variable-Encadenado ::= "idMétodoVariable" EncadenadoOp {AccesoVar.nodo = new AccesoVarCadena(var: new Var(idMetodoVariable.lex), cadena: EncadenadoOp.nodo)}
| "idMétodoVariable" "[" Expresión "]" {AccesoVar.nodo = new AccesoArray(var: new Var(idMetodoVariable.lex), tamaño: Exp.nodo)}

EncadenadoOp ::= Encadenado
| λ {EncadenadoOp.nodo = Encadenado.nodo}

Validaciones

A continuación se listan una serie de consideraciones y decisiones de diseño semántico relevantes antes de profundizar en la implementación.

- No se permite declarar con el mismo identificador dos atributos, dos métodos, dos argumentos de un método o dos variables dentro de un bloque de método.
- No se permite declarar una variable en el bloque de un método con el mismo nombre que un argumento del método, ya que impediría el acceso al argumento.
- No se permite declarar variables, argumentos, atributos ni métodos utilizando el identificador “self”.
- Sí se permite declarar variables, argumentos y atributos utilizando el identificador “create”. Pero no se permite declarar métodos utilizando dicho identificador.
- Se agregó en la etapa léxica la validación en tiempo de compilación que los literales de tipo I32 no excedan los 32 bits.
- En la tabla de símbolos se guarda registro de las variables de cada bloque de método para su posterior uso por el AST durante sus validaciones.
- Se valida para cada método que el bloque retorna el tipo de dato esperado para el método. Si dentro del bloque hay sub-bloques o ramas, se valida que en cada rama se retorne el tipo esperado.
- Se permite usar instancias de una clase dada en contextos donde se espera una

instancia de su superclase. Por ejemplo, al asignar variables o retornar dentro de un método.

- No se permite acceder atributos ni métodos no estáticos de la clase dentro de métodos estáticos.
- No se permite el acceso a atributos privados de una clase en otras clases ni en subclases. Pero **sí** se permite el acceso a atributos privados de otras instancias de la misma clase, comportamiento similar a Java. Por ejemplo, si el constructor de la clase Nodo recibe una instancia distinta de la clase Nodo como argumento, los atributos privados de esa instancia sí son accesibles:

```
class Nodo {
    I32: attrPrivado;
    public Nodo (Nodo: ejemploVar) {
        self.attrPrivado = ejemploVar.attrPrivado; // EXITO.
    }
}
```

- Sí se permite invocar de manera dinámica a métodos estáticos. Este es un comportamiento similar al de Java. Si bien Java da una advertencia al respecto, no es un error. Un ejemplo válido sería:

```
class Ejemplo {
    static fn metodoEstatico() -> void {}
    fn metodoDinamico(Ejemplo: var) -> void {
        Ejemplo.metodoEstatico();  // EXITO.
        metodoEstatico() // EXITO.
        (var.metodoEstatico()) // EXITO.
    }
}
```

Errores

En la etapa de análisis semántico de **declaraciones** pueden lanzarse distintos errores de chequeo semántico durante el análisis y construcción de la TS y durante la posterior etapa de consolidación. En la implementación se creó una clase para cada uno de estos errores que extiende de la clase abstracta *SemanticalError*. Cada tipo de error se lista a continuación:

- *Constructor mal formado*: error generado al encontrar una declaración de un método de clase cuyo identificador es “create”. El error sugiere la sintaxis apropiada para declarar el constructor, ya que es ligeramente diferente de la sintaxis normal de declaración de métodos.
- *Múltiples constructores*: error generado al encontrar dos constructores declarados en una misma clase, ya que el lenguaje soporta un único constructor por clase.
- *Entidad duplicada*: error generado al declarar dos clases o dos métodos de una misma clase o dos argumentos/variables de un mismo método con el mismo identificador.
- *Herencia base ilegal*: error generado al hallar una clase que herede de cualquiera de las clases predefinidas de TinyRust+, estas son: Str, Char, I32, Bool y Array.
- *Herencia circular*: error generado durante la etapa de consolidación si una clase hereda

directa o indirectamente de sí misma.

- *Clase inexistente*: error generado durante la etapa de consolidación si una clase hereda de otra clase no declarada o si un atributo, argumento de método, retorno de método o variable de método son de una clase no declarada.
- *Error de consolidación*: todo error generado durante la etapa de consolidación que no se haya mencionado previamente se cataloga en este grupo. Estos errores tienen que ver con la redefinición de entidades de una superclase en la subclase. El error incluye información explicativa para el usuario sobre el escenario en que ocurrió. Algunos escenarios posibles de este error son si se redefinió un método estático o atributo de una superclase o se redefinió un método de una superclase con distinta firma.

En el análisis semántico de **sentencias** pueden aparecer errores durante la etapa de validación del AST, pero no durante su construcción, dado que la sintaxis y las declaraciones ya están validadas para cuando se construyen los nodos del AST. En la implementación todos estos errores son de clase *ASTError*, que extiende de *SemanticalError*. Los distintos errores de validación se listan a continuación:

- *Expresión de tipo incorrecto*: error generado al recibir una expresión que no es del tipo esperado ni un subtipo para el contexto en que se está usando. Por ejemplo, si en la condición de un if se utiliza una expresión no booleana, si se utilizan operandos de tipo string al realizar una operación de resta o similares, si se retorna en un método una expresión que no coincide con el tipo de retorno del método o si se asigna en una variable un valor de un tipo distinto al declarado para la variable
- *Entidad inaccesible*: error generado al intentar acceder a una variable, método o atributo que no están definidos para la clase actual o no son visibles. Este caso se puede dar al intentar acceder variables del método actual, al intentar acceder atributos (privados o públicos) y métodos de la clase actual, al invocar métodos estáticos, en los accesos encadenados tanto de métodos como atributos, al acceder la referencia self dentro de un método estático o al acceder atributos de instancia en un método estático.
- *Invocación inválida de método*: error generado cuando la invocación de un método no respeta la definición formal del método. Ocurre al invocar de forma dinámica un método estático de una clase o viceversa, al invocar al método create explícitamente y al invocar el método con una cantidad incorrecta de argumentos o si sus tipos no coinciden con la definición formal.
- *Invocación inválida de arreglo*: error generado al intentar acceder un índice de un atributo de una clase cuando el atributo no es de tipo arreglo.

Validaciones semánticas

En la sección previa ya se explicaron brevemente los tipos de error que pueden ocurrir durante el análisis semántico. En esta sección se explicará con mayor detalle cada validación y el momento en que se realiza. Este último dato es importante, ya que de esto depende en qué parte del código se implementa la validación

Los chequeos semánticos de declaraciones ocurren en dos momentos: durante la construcción de la tabla de símbolos y durante la consolidación de la tabla.

Las validaciones durante la construcción de la tabla de símbolos se realizan al invocar alguno de los métodos públicos de *SymbolTable,* si la validación falla, se lanza una excepción. Estas excepciones son elevadas en última instancia por *SyntacticAnalyzer* (quien invoca a *SymbolTable*) e interceptadas por el *Executor*. Durante la construcción de la tabla de símbolos se realizan los siguientes chequeos:

1. Al guardar una **clase** en la TS, se valida que no se haya guardado previamente una clase con el mismo nombre.
2. Al guardar una **clase** en la TS, se valida que no herede de una clase base o predefinida de TinyRust+.
3. Al guardar el **constructor** de una clase en la TS, se valida que no haya guardado previamente otro constructor para esa misma clase, dado que TinyRust+ no soporta múltiples constructores por clase.
4. Al guardar un **atributo** de una clase en la TS, se valida que no se haya guardado previamente otro atributo en esa misma clase con el mismo nombre.
5. Al guardar un **atributo** de una clase en la TS, se valida que su nombre no sea “self”, ya que colisionará con la referencia a la instancia de la clase.
6. Al guardar un **método** de una clase en la TS, se valida que no se haya guardado previamente otro método en esa misma clase con el mismo nombre.
7. Al guardar un **método** de una clase en la TS, se valida que su nombre no sea “self”, ya que colisionará con la referencia a la instancia de la clase.
8. Al guardar un **método** de una clase en la TS, se valida que su nombre no sea “create”, ya que en este caso debería declararse el método con la sintaxis del constructor (que no incluye el tipo de retorno ni la palabra reservada “fn”).
9. Al guardar un **argumento** de un método de una clase en la TS, se valida que no se haya guardado previamente otro argumento en ese mismo método con el mismo nombre.
10. Al guardar un **argumento** de un método de una clase en la TS, se valida que su nombre no sea “self”, ya que colisionará con la referencia a la instancia de la clase.
11. Al guardar una **variable** del bloque de un método en la TS, se valida que no se haya guardado previamente otra variable o argumento en ese mismo método con el mismo nombre.
12. Al guardar una **variable** del bloque de un método en la TS, se valida que su nombre no sea “self”, ya que colisionará con la referencia a la instancia de la clase.

Las validaciones de consolidación de la tabla de símbolos se realizan (luego de que la TS haya sido construída por completo) dentro del método *consolidate* de *SymbolTable* o en el método homónimo de cualquiera de sus entradas anidadas. Si la validación falla, se lanza una excepción. Estas excepciones son elevadas por *SymbolTable* e interceptadas directamente por el *Executor,* que es quien invocó al método *consolidate* de *SymbolTable*. Durante la etapa de consolidación de la tabla de símbolos se realizan los siguientes chequeos semánticos:

1. Para cada **clase** guardada en la TS se valida que su superclase esté definida en la tabla (a excepción de las clases Object y main, que no poseen superclase).
2. Para cada **clase** guardada en la TS se valida que no posea herencia circular, es decir,

que no herede directa o indirectamente de sí misma.

3. Para cada **atributo** de una clase, se valida que su tipo coincida con una clase definida en la TS.
4. Para cada **atributo** de una superclase, se valida que la subclase no defina un atributo con el mismo nombre, ya que TinyRust+ no soporta redefinición de atributos de ningún tipo.
5. Para cada **método estático** de una superclase, se valida que la subclase no defina un método con el mismo nombre, ya que TinyRust+ no soporta redefinición de métodos estáticos.
6. Para cada **método no estático** de una superclase, se valida que la subclase no defina un método con el mismo nombre y distinta firma o modificador, ya que TinyRust+ no soporta redefinición de métodos con distinta firma. En este caso se valida que coincidan el tipo de retorno y la cantidad, orden y tipo de argumentos formales para que el método se considere correctamente redefinido.
7. Para cada **argumento** y **tipo de retorno** de un método se valida que su tipo coincida con una clase definida en la TS.
8. Para cada **variable** en el bloque de un método se valida que su tipo coincida con una clase definida en la TS.

Las validaciones semánticas de sentencias ocurren luego de que se hayan construido por completo el AST y la tabla de símbolos esté completa y consolidada. En la implementación, estas validaciones ocurren dentro del método *validate* de algún nodo del AST. Si la validación falla, se lanza una excepción que es elevada por *AbstractSyntaxTree* e interceptada directamente por el *Executor,* que es quien invocó al método *validate* en el nodo raíz del AST. Las validaciones de sentencias que se efectúan sobre el AST son las siguientes:

1. Para cada **método** se valida que el tipo retornado por el bloque coincida con el tipo de retorno formal del método.
2. Para cada **método estático** se valida que no se acceda a la referencia self ni a atributos o métodos de instancia.
3. Para cada **bloque de método**, se verifica que todas sus ramas o bloques condicionales anidados retornen un tipo de dato que coincida o sea una subclase del tipo de retorno del método.
4. Para cada **sentencia de asignación** se valida que el tipo de la expresión del lado derecho sea el mismo o una subclase del tipo de la expresión del lado izquierdo.
5. Para cada **sentencia if** se valida que el tipo de la expresión condicional sea booleano.
6. Para cada **sentencia while** se valida que el tipo de la expresión condicional sea booleano.
7. Para cada **sentencia return** se valida que el tipo de la expresión retornada coincida con el tipo de retorno formal del método o sea una subclase de esta.
8. Para cada **expresión** se valida que el tipo retornado sea el esperado para el contexto en el cuál es usada.
9. Para cada **expresión binaria** se valida que ambos operandos sean del mismo tipo, En el caso de las operaciones aritméticas, se valida que ambos operandos sean de tipo I32.

En las operaciones lógicas se valida los operandos sean de tipo booleano. Mientras que en las operaciones *!=* y *==* no se exige ningún tipo particular.

10. Para cada **expresión unaria** se valida que el operando sea del tipo correcto para el operador. En caso de los operadores de signo se espera que el operando sea de tipo I32, mientras que para el operador de negación se espera que el operando sea de tipo booleano.
11. Para cada **acceso a variable** se valida que exista una variable, argumento o atributo visible en el contexto actual con tal identificador. En el caso de los atributos, se valida que el atributo no sea privado heredado, ya que no sería visible en tal caso.
12. Para cada **acceso a método dinámico** se valida que la clase actual posea un método con tal nombre y que la cantidad, tipo y orden de argumentos coincida con la definición formal.
13. Para cada **acceso a método estático** se valida que la clase actual posea un método estático con tal nombre y que la cantidad, tipo y orden de argumentos coincida con la definición formal.
14. Para cada **acceso a atributo encadenado** se valida que la clase accedida posea un atributo visible con tal nombre. Dentro de la misma clase, son visibles todos los atributos excepto los privados heredados. Fuera de la clase, son visibles únicamente los atributos públicos.
15. Para cada **acceso a método encadenado** se valida que la clase accedida posea un método con tal nombre y que la cantidad, tipo y orden de argumentos coincida con la definición formal.
16. Para cada **acceso a elemento de un arreglo (simple o encadenado)** se valida que la entidad accedida sea de tipo arreglo y que la expresión del índice sea de tipo I32.
17. Para cada **llamada a constructor** se valida que la cantidad, tipo y orden de argumentos coincida con la definición formal.
18. Para cada **llamada a constructor de arreglo** se valida que el tamaño del arreglo sea un literal entero. Para simplificar el análisis, no se aceptan expresiones para asignar el tamaño de un arreglo, es decir, el tamaño de los arreglos debe conocerse estáticamente en tiempo de compilación.

Implementación

En esta sección se ahondará en la implementación del analizador semántico de TinyRust+, explicando el rol, funcionamiento y métodos de cada una de las clases que lo componen.

Executor

Esta clase consta de un único método llamado *run*. Al invocar este método, se crea una instancia de la clase *SyntacticAnalyzer* que recibe como parámetro la ruta del archivo de entrada. A su vez la ruta de entrada se utiliza para construir el nombre de la ruta de los archivos de salida de la TS y el AST cambiando únicamente la extensión del archivo, pero conservando el nombre.

Una vez instanciado el analizador, el ejecutor llamará al método *run* del *SyntactycAnalyzer*, el cual realiza el análisis sintáctico y construye la tabla de símbolos y

el AST. Cuando este método retorna, tanto el AST como la tabla de símbolos se encuentran construidos. En este caso, el ejecutor invoca al método consolidate de la tabla de símbolos y posteriormente al método validate del AST. En caso de completar la ejecución de ambos métodos, la TS estará ya unificada y el AST también estará completo y validado. En tal caso, un mensaje de éxito se muestra por pantalla y se escriben dos archivos: una representación en formato JSON de la tabla de símbolos se escribe en un archivo con extensión *.ts.json* y una representación JSON del árbol sintáctico abstracto se guarda en un archivo con extensión *.ast.json*. Si el archivo no fuera sintáctica o semánticamente correcto, *Executor* intercepta la excepción lanzada por *SyntacticAnalyzer* y en la salida muestra línea y columna donde se presentó el error, incluyendo en la descripción si se trata de un error léxico, sintáctico o semántico. Si el archivo contuviera declaraciones o sentencias inválidas semánticamente para TinyRust+, el método run lanzará una excepción de tipo *SemanticalError*, que será interceptada y escrita en el medio de salida, incluyendo línea, columna y descripción del error.

También existe la implementación de una clase *Executor* secundaria que sólo llega hasta el punto de consolidar la tabla de símbolos antes de retornar. Esta implementación sirve para poner a prueba de forma aislada el funcionamiento de la tabla de símbolos, sin preocuparse por las validaciones del AST.

SymbolTable

Esta clase implementa las responsabilidades de la tabla de símbolos. En sus atributos guarda registro del nombre del archivo de entrada, la lista de clases definidas, y la clase y el método analizados actualmente.

Para inicializar la clase únicamente se debe invocar su constructor proporcionando el nombre del archivo. En su lista de clases incluirá por defecto todas las clases predefinidas de TinyRust+.

Los atributos *currentClass* y *currentMethod* guardan el estado actual de análisis de la entrada tanto durante la construcción de la TS como durante la segunda pasada de validación del AST. Durante la construcción de la TS, ambas variables facilitan la tarea de agregar métodos, atributos, argumentos y variables a las clases/métodos de la tabla, ya que todas estas entidades siempre son agregadas al método o clase actual. Durante la etapa de validación del AST, estas variables de estado guardan registro de qué clase y método se están validando actualmente. Esto facilita la tarea de obtener una variable o método de la TS para validar una expresión guardada en el AST.

Posee dos métodos relevantes que son: *consolidate* y *toJson*.

consolidate: es el método que realiza el proceso de consolidación de la tabla una vez que ya se ha consumido todo el archivo de entrada. El método se invoca sin parámetros y no retorna ningún valor. En este procedimiento se revisa una a una cada clase declarada y se realizan dos pasos. En una primera instancia, se valida que la clase herede de una clase existente (posiblemente Object) y que no exista herencia circular. Si no pasa la validación, el método lanza una excepción. En caso de pasar esta primera

validación, se llama al método *consolidate* de la clase pasándole su lista de ancestros para que se agreguen los atributos y métodos heredados a la clase. Luego de consolidar una clase, se marca un flag de "consolidada" de modo que si otra clase hereda de esta, no se realice de nuevo el proceso de consolidación de esta clase hacia sus ancestros. Se realiza una optimización almacenando las relaciones de herencia en un árbol de ancestros. Cuando una clase se agrega al árbol, está garantizado que está consolidada y no tiene herencia cíclica. De modo que al ser referenciada posteriormente, si ya está en el árbol, no se repite todo el proceso de validación y se usa directamente.

toJson: se invoca sin parámetros y devuelve un *String* conteniendo la representación en formato JSON de la tabla y su contenido. Debe invocarse cuando la tabla ya ha sido construida por completo y consolidada. Caso contrario, la información estará incompleta. El JSON se construye de forma recursiva al invocar el método homónimo en los distintos atributos de la tabla.

La clase posee también una serie de métodos para agregar distintos tipos de entidades a la tabla de símbolos:

addClass: recibe un identificador y agrega a la lista de clases declaradas una nueva entrada de clase con ese identificador. Actualiza la referencia a la clase actual apuntando a esta nueva clase, de modo que al agregar métodos, constructores y atributos posteriormente, se agreguen en esta clase.

addMethod: recibe un identificador y un booleano indicando si el método es de clase o instancia para agregar una nueva entrada a la lista de métodos de la clase analizada actualmente, establecida previamente en el método *addClass*.

addMain: agrega una "clase fantasma" a la lista de clases de la tabla. Esta clase fantasma representa al método main. No es heredable y no posee métodos, atributos ni constructor.

addConstructor: inicializa un constructor en la clase analizada actualmente y registra al constructor como el método analizado actualmente. A medida que el análisis avance, se agregarán los parámetros formales al constructor.

addVar: recibe un identificador, un tipo de dato de TinyRust+ y un booleano indicando si la variable es pública. Si actualmente se está analizando una clase pero no un método, esto indica que la variable es un atributo de la clase, por lo cual se llama al método *addAttribute* de la clase analizada. En caso de que se estuviera analizando un método, la variable no se registra, ya que indicaría que es una variable declarada dentro del bloque del método y no es una entidad relevante en esta etapa de análisis.

SymbolTable también provee algunos métodos para facilitar datos al AST como *getVariable, getClas*s e *isSubclas*s. Mientras que las primeras dos funciones devuelven entidades de la TS, la tercera valida si una clase es subclase de otra clase dada.

Entidades de la Tabla de Símbolos

Para cada entidad de la Tabla de Símbolos, se creó una clase distinta que implementa su comportamiento.

ClassEntry

La clase *ClassEntry* representa una clase declarada en un archivo de TinyRust+. Entre sus atributos guarda: nombre de la clase, nombre de la superclase, referencia al constructor, booleano indicando si es heredable, lista de atributos y lista de métodos. A su vez posee un flag booleano indicando si la clase ya ha sido consolidada. Al ser una entidad de la tabla de símbolos, implementa el método *toJson*.

Posee métodos para agregar un nuevo atributo, un nuevo método, modificar el constructor y el tipo de retorno y setters/getters para acceder y modificar los distintos atributos de la instancia. Cada uno de estos métodos puede arrojar excepciones si no se respeta la semántica de TinyRust+, por ejemplo, si se intenta agregar dos métodos o dos atributos con el mismo identificador, si se intenta agregar un método o atributo con el identificador “self”, si se declaró más de un constructor para la clase de TinyRust+ o si se declaró el constructor con sintaxis de método común.

El método más relevante de esta clase es *consolidate*, que realiza la consolidación de la clase de TinyRust+ al ser invocado. Este método es invocado únicamente por *SymbolTable* u otra instancia de *ClassEntry* durante la consolidación y recibe el árbol de ancestros completo (ya que previo a la consolidación sólo se conoce el identificador de la superclase, pero no se tiene una referencia concreta). En caso de que la clase no haya sido previamente consolidada y herede de una superclase (la clase *Object* y la clase fantasma *main* heredan de *null*, por lo que la validación es necesaria), se procede a realizar el proceso de consolidación. Como se mencionó anteriormente, *ClassEntry* posee un atributo booleano que indica si la clase ya fue consolidada, de este modo se evita realizar procesamiento innecesario al pasar por el proceso de consolidación más de una vez para la misma clase.

El proceso de consolidación comienza invocando al método *consolidate* de la superclase recursivamente, pasándole el árbol de ancestros. Cuando la superclase ya está consolidada, se realiza la consolidación de atributos y métodos de la clase actual. Cada atributo de la superclase se agrega a la lista de atributos de la subclase solo si esta no posee un atributo con el mismo identificador. En caso de colisión, se lanza una excepción, dado que no se permite redefinir atributos en TinyRust+. Los métodos también se heredan directamente si no hay colisión de identificadores. En caso de que la subclase redefina un método, el método de la superclase **no** se agrega a la subclase y se valida que su firma coincida. En esta validación se comprueba que el método no sea estático y que coincidan el tipo de retorno, la cantidad de argumentos formales y tipo y orden de los argumentos. En caso de no pasar la validación, se lanza una excepción de tipo *ConsolidationError* con un mensaje explicativo al usuario. Tras consolidar métodos y atributos exitosamente, se actualiza a *true* el booleano que indica que la clase ya ha sido

consolidada y finaliza la ejecución del método.

MethodEntry

Esta clase representa un método de una clase definida en TinyRust+. Incluye distintos atributos que registran el nombre del método, su posición, tipo de retorno, lista de argumentos y un booleano indicando si es un método de instancia o de clase.

Al ser una entidad de la tabla de símbolos, implementa el método *toJson*, que genera la representación JSON de únicamente el método. Posee una serie de setters y getters para acceder y modificar sus atributos. Además de estos, posee el método *addArgument* para agregar un nuevo argumento a la lista de argumentos de la instancia. Para agregar un argumento se debe invocar al método con el identificador y tipo del nuevo argumento, que puede arrojar una excepción si el identificador es “self” o ya existe otro argumento con el mismo identificador.

ConstructorEntry

La clase *ConstructorEntry* hereda de *MethodEntry* y representa el constructor de una clase de TinyRust+. Dado que la lógica genérica de todo método está implementada en la superclase *MethodEntry*, esta clase sólo implementa una versión simplificada del método *toJson* para el constructor y se guarda registro de si el constructor fue declarado de forma explícita o es el constructor implícito por defecto. Este valor se consulta a la hora de validar si una clase de TinyRust+ posee más de un constructor declarado.

VariableEntry

Esta clase representa una variable del lenguaje TinyRust+. En sus atributos guarda el nombre, tipo y posición de la variable. Implementa getters y el método *toJson*. No es una entidad utilizada explícitamente en la tabla de símbolos, pero *AttributeEntry* y *ArgumentEntry* heredan de esta clase.

AttributeEntry

AttributeEntry representa una variable del lenguaje TinyRust+ y hereda de la clase *VariableEntry*. Únicamente agrega el registro de si el atributo es público o privado, dado que el tipo, valor y posición los guarda la superclase. Implementa el método *toJson* incluyendo todos estos datos en la representación JSON.

ArgumentEntry

AttributeEntry representa un argumento formal de un método en el lenguaje TinyRust+ y hereda de la clase *VariableEntry*. No agrega ninguna funcionalidad a la superclase, su valor es meramente semántico.

T**ipos Predefinidos**

La clase abstracta *PredefinedClass* representa una clase predefinida de TinyRust+.

Extiende de *ClassEntry* y es a su vez heredada por las clases *Array, Bool, Char, I32, IO*, *Object* y *Str*. Todas estas clases concretas representan cada una a una clase predefinida distinta de TinyRust+ y son agregadas como entidad a la lista de clases de la tabla de símbolos durante su inicialización. *PredefinedClass* redefine el método *consolidate* con el tope recursivo. Ya que todas las clases predefinidas heredan de *Object* (o es la clase *Object* propiamente), no se modifican al consolidarse, por lo cual se consideran consolidadas por defecto. Es decir, el método *consolidate* de esta clase únicamente actualiza a true el booleano indicando que la clase ya está consolidada y retorna luego de esto. Todas las clases predefinidas se inicializan con su constructor en *null* y como no heredables.

Cada clase que hereda de *PredefinedClass* debe llamar al constructor de *PredefinedClass* con el identificador del tipo (es decir, el nombre de la clase en TinyRust+). Cada una de estas clases puede llamar en su constructor a los métodos de la superclase para registrar la serie de métodos que la clase predefinida de TinyRust+ implementa. Por ejemplo, la clase *Str* registra los métodos *length, concat y substr*; las clases *Array* e *IO* también implementan sus propios métodos.

AbstractSyntaxTree

La clase *AbstractSyntaxTree* representa el nodo raíz del AST del archivo de entrada. Entre sus atributos guarda una lista de nodos de clase (*ClassNode*). Cuando es instanciada, inicializa esta lista y agrega un nodo para cada clase predefinida de TinyRust+ a modo de *placeholder*.

Esta clase se instancia en *SyntacticAnalyzer* antes de comenzar el análisis. Durante el análisis sintáctico, se irán generando instancias de *ClassNode* a medida que se encuentren declaraciones de clases de TinyRust+ en el archivo de entrada. Dado que el AST se construye desde las hojas hasta la raíz, cuando un *ClassNode* se agrega a la lista, esto implica que todo su subárbol ya ha sido construido, por lo cual no se está agregando un solo nodo al AST, sino una rama completa del árbol. Se incluye en la lista también un nodo de tipo *ClassNode* representando al método main, esta "clase" posee un único método llamado de igual forma que guarda todas las sentencias de main.

A la hora de validar el AST, la validación recursiva comienza invocando al método *validate* de *AbtractSyntaxTtee* con la tabla de símbolos ya consolidada como único argumento. Este método recursivo únicamente invoca al método homónimo de cada *ClassNode* de la lista de clases. Si todos los *ClassNodes* pasan la validación, el AST completo se considera válido. El método *validate* de cualquier nodo espera recibir la tabla de símbolos ya consolidada (una instancia de *SymbolTable*) y lanza una excepción de tipo *ASTError* en caso de que el nodo o su subárbol no sean válidos semánticamente. Si *validate* completa su ejecución, implica que el nodo y todo su subárbol son válidos.

Para cada tipo de nodo distinto del AST se implementó una clase. A continuación se explica la composición y funcionamiento de los principales tipos de nodos.

ClassNode

Una instancia de *ClassNode* representa en el AST una clase de TinyRust+ declarada en el archivo de entrada. En sus atributos guarda únicamente el nombre de la clase y una lista *MethodNodes* representando a cada método declarado para esta clase. En el AST no se guarda registro de los atributos de cada clase, ya que esta información se encuentra en la TS. Si la clase posee la declaración de un constructor, se incluirá una entrada de método extra en la lista, con el nombre "create" y la información del constructor, de modo que sus sentencias puedan ser validadas.

Cuando se instancia un elemento de esta clase, recibe como argumentos el nombre de la clase y la lista de métodos ya construida por completo. A partir de entonces, el elemento no se modifica, únicamente se valida posteriormente.

La validación de este nodo consiste en validar cada uno de los métodos de la lista. Para esta validación, se modifica el estado de la TS actualizando *currentClass* con la entrada de clase (de la TS) correspondiente a medida que se analiza cada clase. Esto es para poder acceder en la TS a los métodos y atributos locales de la clase analizada sin que cada nodo del árbol deba saber explícitamente el nombre de la clase. Una clase se considera válida si todos sus métodos pasan la validación.

MethodNode

Una instancia de *MethodNode* representa en el AST un método o constructor de una clase de TinyRust+. En sus atributos guarda el nombre del método y un *BlockNode*, una referencia al bloque de sentencias del método. Tanto los argumentos del método como tipo de retorno y declaración de variables del bloque se encuentran ya guardados en la TS, por lo cual no se agregan a este nodo del AST.

Cuando se instancia un elemento de esta clase, espera recibir el nombre de la clase y el bloque de sentencias ya construído. Si el método pertenece a una clase predefinida, se agrega el flag *mocked* indicando que no se deben realizar validaciones sobre este nodo. Esto es debido a que los métodos predefinidos son únicamente un *placeholder* y no poseen sentencias, por lo cual no suelen respetar el tipo de retorno esperado para su bloque y no pasarían la validación.

El método *validate* de MethodNode modifica el estado de la TS actualizando el valor de *currentMethod* con la entrada de la TS del método en cuestión. De este modo las variables del bloque y argumentos formales son accesibles para las validaciones de sentencias del bloque sin necesidad de que cada nodo sepa explícitamente el nombre del método. La validación del método contempla dos factores: primero, que el bloque sea válido (es decir, que cada sentencia del bloque sea válida) y segundo, que el bloque tenga un tipo de retorno que coincida con el tipo de retorno declarado para el método.

SentenceNode

Esta es una clase abstracta que engloba a los distintos tipos de sentencias de

TinyRust+. De esta clase heredan: *BlockNode*, *AssignNode*, *SimpleExpressionNode*, *IfElseNode*, *WhileNode* y *ReturnNode*.

SentenceNode guarda entre sus atributos el tipo esperado de retorno de la sentencia y el tipo real retornado por la sentencia. Estos atributos son utilizados para validar que todas las ramas del bloque de un método retornen el tipo correcto.

La clase *BlockNode* representa un bloque de sentencias de TinyRust+. Durante la etapa de validación, este nodo invoca al método *validate* de cada una de sus sentencias y se considera válido si todas sus sentencias son válidas. Si alguna de sus sentencias tiene un tipo de retorno, entonces determina el tipo de retorno del bloque.

La clase *AssignNode* representa una asignación de una expresión a una variable. Posee dos atributos: *leftSide* de tipo *AccessNode* y *rightSide* de tipo *ExpressionNode*. En este nodo se valida que el tipo de la expresión evaluada del lado derecho coincida con el tipo del nodo accedido del lado izquierdo de la asignación.

La clase *SimpleExpressionNode* representa una sentencia simple y guarda únicamente una expresión. El método *validate* de esta sentencia se encarga de llamar al método *validate* de la expresión.

La clase *IfElseNode* representa una sentencia *if/else* de TinyRust+. Entre sus atributos guarda un *ExpressionNode* representando la condición, una *SentenceNode* representando la sentencia condicional y opcionalmente una segunda sentencia del bloque *else*. Su método *validate* corrobora que la expresión condicional sea válida y resuelva un booleano, mientras que valida recursivamente la sentencia del bloque condicional y el bloque *else* si existe. Si las sentencias de ambas ramas retornan un valor, entonces la sentencia *IfElseNode* tendrá el mismo tipo de retorno, caso contrario queda indefinido.

La clase WhileNode representa un bucle *while* de TinyRust+. En sus atributos guarda un *ExpressionNode* representando la condición y una *SentenceNode* representando la sentencia del bucle. Su método *validate* corrobora que la expresión condicional sea válida y resuelva un booleano, mientras que valida recursivamente la sentencia del bloque condicional. Si la sentencia del bucle retorna un tipo de valor, entonces la sentencia *WhileNode* tendrá el mismo tipo de retorno, caso contrario queda indefinido.

La clase *ReturnNode* representa una sentencia return de TinyRust+. Este nodo guarda un *ExpressionNode* representando la expresión de retorno. En caso de no retornar ningún valor o retornar *nil*, la expresión es de tipo *Void().* Su método *validate* corrobora que la expresión sea válida y el tipo resuelto de la expresión coincida con el tipo de retorno esperado para el método actual. El tipo de retorno de esta sentencia es, justamente, el mismo tipo resuelto por la expresión. Este tipo es el que luego determinará el tipo de la sentencia *BlockNode* que la contiene.

ExpressionNode

ExpressionNode es una clase abstracta que engloba a los distintos tipos de expresiones de TinyRust+. La característica principal de este tipo de nodo es que todas sus subclases deben resolver un valor. De esta clase heredan: *BinaryExpressionNode, UnaryExpressionNode, LiteralNode, ChainedExpressionNode, AccessNode, ConstructorArrayCallNode* y *MethodCallNode.*

Esta clase abstracta guarda dos atributos de tipo *Type*: *expectedResolveType* y *resolveType*, respectivamente son el tipo que se espera que la expresión resuelva (según su contexto) y el tipo formal efectivamente resuelto por la expresión tras validarla. El método *validate* debe ser implementado por sus subclases concretas, pero *ExpressionNode* provee el método *validateType* para corroborar que el tipo de dato esperado y el resuelto coinciden. Si no hay tipo esperado o el tipo resuelto es subclase del esperado entonces la validación también pasa.

La clase *BinaryExpressionNode* representa una operación binaria ya sea lógica o aritmética. Guarda dos atributos de tipo *ExpressionNode* representando ambos operandos y un atributo de tipo *String* que guarda el operador. Cuando se instancia esta clase se debe especificar el tipo que retorna la operación y cuál es el tipo esperado para los operandos si los hay. Al invocar el método *validate* de esta clase se validan ambos operandos y se verifica que el tipo de ambos coincida con el esperado.

La clase *UnaryExpressionNode* representa una operación unaria ya sea lógica o aritmética. Guarda un atributo de tipo *ExpressionNode* representando el operando y un atributo de tipo *String* que guarda el operador. Cuando se instancia esta clase se debe especificar el tipo que retorna la operación y cuál es el tipo esperado para el operando. Al invocar el método *validate* de esta clase se valida la expresión del operando y se verifica que el tipo resuelto coincida con el esperado.

La clase *LiteralNode* representa un literal de TinyRust+ ya sea entero, cadena, caracter o booleano. Cuando se instancia espera recibir el valor del literal y su tipo. Guarda únicamente un elemento de tipo *PrimitiveType* que contiene tanto el tipo como el valor del literal. Su método *validate* únicamente valida que el tipo del literal sea el esperado para su contexto llamando a *validateType* de la superclase *ExpressionNode*. Las instancias de esta clase suelen ser las hojas del AST, por lo tanto, son de los primeros nodos en generarse.

La clase abstracta *AccessNode* representa el acceso a una variable local del contexto actual, ya sea del método o clase actual. Estas expresiones suelen formar parte del lado izquierdo de una asignación o ser el primer nodo de un encadenado. Las clases *AccessArrayNode* y *AccessVariableNode* heredan de esta clase, aunque son muy similares. La primera guarda el nombre de la variable y una expresión del índice accedido, mientras que la segunda guarda el nombre de la variable y un encadenado opcional. El método *validate* de ambas subclases es similar: se obtiene la entidad de la TS, se valida que exista y sea del tipo esperado, en caso de ser un atributo se valida que no se acceda en un contexto estático. Si es un arreglo, se valida que la expresión del índice sea de tipo I32; si es una variable común, se valida el encadenado si existe. Si se

accedió un atributo se valida que sea visible en ese contexto. Si se accedió la referencia self, se valida que no se esté reasignando. Luego de resolver el tipo de retorno, se valida que este tipo coincida con el tipo esperado para su contexto.

La clase *ChainedExpressionNode* guarda una expresión de tipo *ExpressionNode* junto con un acceso encadenado de tipo *ChainedAccessNode*. Esta clase se usa para representar distintos tipos de expresiones que poseen un encadenado, como primario encadenado, expresión parentizada y llamadas encadenadas a métodos estáticos. El tipo resuelto por esta expresión es el resultado del último acceso de la cadena de encadenados. En su método *validate* simplemente se valida la expresión y se resuelve el tipo del encadenado recursivamente.

La clase *MethodCallNode* es una abstracción para distintos tipos de accesos a un método, como pueden ser acceso a un método de instancia, acceso a un método estático o llamada al constructor de una clase. Guarda el nombre del método junto con una lista de *ExpressionNodes* representando los argumentos del método y un *ChainedAccessNode* representando un encadenado opcional. El método *validate* de esta clase realiza varias validaciones: que el método exista en el contexto dado y que la cantidad y tipo de cada argumento coincida con la definición formal del método, que el encadenado sea válido y que el tipo resuelto para el método junto con el encadenado sea el esperado para su contexto. El tipo resuelto por esta expresión es igual al tipo retornado en última instancia por el encadenado.

La clase *ConstructorArrayCallNode* representa el llamado al constructor de un arreglo. Guarda un atributo de tipo *PrimitiveType* representando el tipo de dato de los elementos del arreglo y un nodo de tipo *ExpressionNode* conteniendo la expresión del tamaño del arreglo. Su método *validate* simplemente valida que el tipo de la expresión de tamaño sea de I32 y que el arreglo sea del tipo esperado para su contexto.

ChainedAccessNode

ChainedAccessNode es una clase abstracta utilizada para construir accesos encadenados. Cada nodo de este tipo representa un acceso a método o atributo en una cadena de accesos y posee una referencia al siguiente nodo de la cadena. El tipo resuelto por una cadena de accesos es igual al tipo resuelto por el último nodo de la cadena.

De esta clase heredan *ChainedAttributeNode*, *ChainedArrayNode* y *ChainedMethodNode*. Cada una representa un tipo distinto de acceso y realiza las validaciones pertinentes. La cadena se valida recursivamente, por lo que si el nodo es el último de la cadena, retorna recursivamente su tipo hasta el primer nodo de la cadena.

ChainedAttributeNode valida que el atributo accedido exista y sea público, para invocar la validación del siguiente nodo si existe, pasándole el tipo del atributo o retorno del método.

ChainedArrayNode guarda también una expresión del índice accedido. Esta clase valida que el atributo accedido exista, sea público y sea de tipo *Array*, mientras que valida que la expresión del índice es de tipo I32. Si existe un siguiente nodo, invoca su validación pasándole el tipo de dato de los elementos del arreglo.

ChainedMethodNode guarda el nombre del atributo y una lista de expresiones representando los argumentos del método. En la validación comprueba que el método exista en la clase y que la cantidad y tipo de argumentos coincidan con la definición formal del método. Si existe un siguiente nodo, invoca su validación pasándole el tipo de dato de retorno del método.

Tipos de Variable

Se implementó la clase abstracta *Type* para representar a los distintos tipos de datos de TinyRust+ a la hora de declarar el tipo de una variable, argumento, atributo o tipo de retorno de un método. Esta clase únicamente posee un atributo que registra el identificador del tipo de dato de TinyRust+. Por ejemplo, “Str”, “Char”, “I32”, etc.

Los elementos de esta clase son utilizados tanto por la tabla de símbolos (para registrar el tipo de sus entidades) como por el AST (para resolver el tipo de expresiones).

Las clases *Array, PrimitiveType, Void* y *ClassType* heredan de *Type*. Las instancias de la clase *Array* incluyen también el tipo de dato contenido en el arreglo (por ejemplo, “Array-Str” o “Array-I32”). El tipo *ClassType* representa a cualquier clase de TinyRust+ definida por el usuario, simplemente guarda el identificador de la clase, pero mantiene la interfaz consistente con *Type*, a diferencia de *ClassEntry*.

De la clase abstracta *PrimitiveType* extienden *Bool, Char, I32* y *Str,* los cuatro tipos primitivos de TinyRust+. Lo único que agrega esta clase intermedia es la posibilidad de especificar el valor por defecto y el valor real de las expresiones literales de este tipo.

Otras clases e interfaces

Para la implementación de la tabla de símbolos de TinyRust+ se crearon también algunas clases e interfaces que no se corresponden directamente con ninguna entidad de TinyRust+, pero que facilitan la extensibilidad y legibilidad del código.

La interfaz *TableElement* es implementada por todos las entidades de la tabla de símbolos y exige que su clase implemente el método *toJson*. El método *toJson* permite construir de manera recursiva la representación en formato JSON de la tabla de símbolos, ya que cada elemento llama al método *toJson* de sus atributos para armar su propia representación.

La interfaz *Node* es implementada por todos los nodos del AST y exige que su clase implemente los métodos *toJson* y *validate*, para garantizar la recursividad de la validación del árbol y generación del JSON.

La clase genérica *TableList* hereda de *HashMap* y sirve para almacenar conjuntos de entidades del mismo tipo de la tabla de símbolos, por ejemplo: lista de métodos, atributos o clases. A su vez, esta clase agrega funcionalidad a *HashMap* ya que implementa también el método *toJson*.

La clase genérica *TreeList* hereda de *LinkedList* y sirve para almacenar listas de nodos del mismo tipo del AST. A diferencia de *TableLIst*, TreeList conserva el orden de sus elementos, pero su complejidad de acceso no es constante. A su vez, esta clase agrega funcionalidad a *LinkedList* ya que implementa también los métodos *toJson* y *validate*, que consisten en llamar al método homónimo de cada uno de sus elementos.

La clase *Location* simplemente guarda un número de línea y columna del archivo de entrada donde se encontró un Token, usada para mostrar correctamente la información en los errores semánticos. Cada entidad de la TS y nodo del AST incluyen un atributo de este tipo.

La clase *InternalError* hereda de *SyntacticalError* es una excepción que indica un error de programación, se utiliza en situaciones donde es prolijo manejar una posible excepción, pero donde teóricamente no es posible que se cumplan las condiciones para lanzar el error. Por ejemplo, si se invocara la función para agregar un argumento al método actual, pero no se estuviera analizando ningún método actualmente.

Generación de código

La etapa de generación de código consiste en traducir la semántica del programa de TinyRust+ a un nuevo programa en algún lenguaje previamente conocido por la computadora. En esta implementación de TinyRust+ se genera código ensamblador para la arquitectura MIPS32. El código generado a partir del AST y la TS es guardado en un archivo con extensión .asm y puede ser ejecutado en una computadora de esta arquitectura o un simulador.

Arquitectura

Siendo TinyRust+ un lenguaje orientado a objetos, hay 3 aspectos de diseño relevantes de la generación de código:

- Representación de clases en memoria: métodos y atributos.
- Implementación del polimorfismo.
- Invocación a subrutinas (métodos).

Para la representación de clases, se utilizan Class Instant Records (CIR, por sus siglas), para la implementación de polimorfismo se utilizan Virtual Method Tables (VT, de forma abreviada) y para el manejo de llamadas a subrutinas se diseñó una arquitectura de registro de activación. En las siguientes secciones se ahondará en cada concepto.

Representación de clases y polimorfismo

La representación de una clase en memoria tiene está formada por dos partes: la representación de los métodos y la representación de los atributos.

Para los métodos de una clase, se genera estáticamente por única vez su código y es accesible mediante un *label*. Para cada clase, se crea estáticamente también su *Virtual Table*, una estructura que contiene de forma ordenada la referencia a cada uno de los métodos de la clase. De esta forma, el método número *n* de una clase se puede invocar accediendo a la dirección en la posición *n* (es decir, con un desplazamiento de n*4) de su *Virtual Table*. Para los métodos heredados de una clase, la referencia apunta directamente al *label* del método de la superclase que lo implementa, el mismo método no se declara múltiples veces cuando se hereda.

Los atributos de una clase se guardan dinámicamente junto con una referencia a su *VT* en un *Class Instant Record*. Los CIR son entidades que se crean en tiempo de ejecución cada vez que se instancia una clase de *TinyRust+*. Cuando un objeto de una clase Ejemplo es instanciado, se crea en memoria (en el *heap*) su *CIR*. La referencia a un *CIR* debe mantenerse en la pila, ya que su *label* es creado dinámicamente y no se puede saber a priori (a diferencia de los *labels* de métodos, por ejemplo). El *CIR* contiene en primer lugar una referencia a la *VT* de la clase y luego de forma ordenada los valores de cada uno de los atributos de la clase, ya sean propios o heredados. Sin importar cuál es el tipo estático declarado para la clase, la *VT* corresponderá a su tipo dinámico en tiempo de ejecución, de modo que la invocación a

métodos se comporte de forma polimórfica, invocando los métodos de la clase dinámica. Cuando se invoca un método *n* de una clase, se accede a su *CIR*, a través de la referencia en su primer posición se accede a la *VT* y en la *VT* se accede a la dirección almacenada en la palabra *n*. Para acceder al atributo en la posición m, se accede al CIR en la palabra m+1 (posición (m+1)*4, por el tamaño de la palabra). A través de esta posición se puede acceder al valor o a una referencia a él (si fuera una cadena o un objeto). El atributo se modifica modificando el valor en esta posición de memoria.

Registro de activación

El registro de activación es un conjunto de datos que se guardan en la pila cuando se invoca a un método y permite: acceder a los argumentos y variables del método, acceder a los atributos y métodos de la clase a la cual pertenece el método (es decir, acceder a *self*) y restaurar el contexto de registros del método invocador cuando la ejecución del método invocado se complete.

En esta implementación de TinyRust+, el registro de activación contiene los siguientes elementos en orden descendente (teniendo en cuenta que la pila crece hacia abajo):

1. Referencia al CIR de la clase. (Para métodos estáticos es nil, ya que no se accede).
2. Return Address del invocador.
3. Frame Pointer del invocador (llamémosle FP1).
4. Argumentos del método en orden inverso: n, n-1,...., 2, 1.
5. Frame Pointer FP1 (llamémosle FP2).
6. Variables del método en orden ascendente: 1, 2,...., m-1,m.

Y los registros quedan configurados de la siguiente manera:

- $fp guarda la dirección de FP2.
- $ra contiene la dirección de la próxima instrucción del invocador.
- $sp apunta a la siguiente posición de memoria luego de la última variable.

Cuando el método completa su ejecución, se borran de la pila todo el registro de activación simplemente apuntando el *Stack Pointer* $sp a la posición de FP1-8 (accesible a través de FP2, que está guardado en $fp). En el registro $a0 se guarda el valor de retorno del método y en los registros $fp, $ra y $sp se restauran los mismos valores que tenían antes de la invocación.

Durante la ejecución del método, los argumentos y variables se acceden a través de $fp. El argumento *n* se accede con un desplazamiento de n*4 hacia arriba del FP, mientras que las variable *m* se accede con un desplazamiento de *m*4* hacia abajo del FP.

Para acceder a métodos y atributos de la clase, se utiliza la referencia al CIR de la instancia que se encuentra al principio del registro de activación, esta se puede acceder a través de $fp que guarda la dirección de FP2, ya que el CIR se encuentra en la dirección FP2 - 8.

Implementación

La implementación de la generación de código se hizo dentro de las clases que componen al AST. El código se genera con la misma táctica recursiva en profundidad usada para la validación del AST. Cada clase que represente un nodo del AST debe implementar el método *generateCode*. Este método recibe la tabla de símbolos como argumento y retorna un String conteniendo código ASM bien formado.

Para comenzar el proceso de generación de código, un ejecutor debe invocar al método *generateCode* del AST(que es una instancia de *AbstractSyntaxTree* e implementa la interfaz de nodo). Este método devolverá un *String* con todo el código ASM del programa. Este String se guarda escribe directamente en el archivo de salida con extensión *.asm*.

Para generar el código, *AbstractSyntaxTree* genera todo el código estático inicial del programa (como constantes y rutinas de limpieza) e invoca al generador de código de los nodos clase, que generan la VT de la clase en el segmento de datos y a su vez se fían del código generado por los nodos método para el cuerpo de cada método. En el cuerpo de los métodos se utiliza el código generado por las sentencias y dentro de las sentencias se utiliza el código generado por los distintos tipos de nodos de expresiones y así sucesivamente.

La clase *AbstractSyntaxTree* es la encargada de generar las primeras instrucciones del segmento de código del ASM propiamente. En este fragmento se hace un salto a la etiqueta *main*, de modo que la ejecución del programa comience con la ejecución del método *main*.

A continuación, se brinda un breve resumen de la naturaleza del código generado por cada tipo de nodo.

AbstractSyntaxTree

- Genera todas las constantes del programa en el segmento de datos.
- Genera el código para la invocación del método main cuando comienza la ejecución.
- Genera subrutinas de manejo de errores, limpieza de registros (al salir de un método).
- Genera la subrutina de terminación del programa (ejecución de la *syscall exit*).

ClassNode

- Genera la VT de la clase en el segmento de datos.
- Genera el código de cada método implementado por la clase, incluyendo el constructor.

MethodCallNode

En la llamada a método, realizada dentro de un bloque de sentencias, se implementa la primera mitad de la generación del registro de activación:

- Cada subclase de *MethodCallNode* (estas son *AccessStaticMethodNode*, *ConstructorCallNode* y *AccessMethodNode*) debe encargarse de poner en la pila la referencia a *self* (el *CIR* de la instancia sobre la cual se invoca el método).

- Esta clase pone en la pila el *Frame Pointer* (FP1) y *Return Address* del invocador.
- Evalúa y pone en la pila los argumentos actuales del método en orden inverso.
- Genera el salto al *label* del método.
- Restaura el *Frame Pointer* y el *Return Address* del invocador al terminar la ejecución del método.

MethodNode

- Genera el label del método.
- Pone en la pila el segundo *Frame Pointer* (FP2).
- Inicializa y pone en la pila las variables declaradas dentro del método en orden ascendente.
- Genera el código del bloque del método o agrega el código predefinido para métodos de las clases predefinidas (como IO, Str y Array).
- Agrega código para un retorno implícito por si no hay retorno explícito en el método.
- Genera un llamado a la subrutina de limpieza de la pila luego del cuerpo del método.

SentenceNode

Cada subclase de *SentenceNode* debe generar el código propio de este tipo de sentencia:

- *AssignNode* genera código para acceder a la entidad de la derecha y guardar en su lugar la expresión evaluada del lado izquierdo.
- *IfElseNode* crea labels para cada bloque, genera el código de cada bloque, evalúa la condición y genera un salto condicional sobre el resultado.
- *WhileNode* crea labels para el principio y final del bucle, evalúa la condición y genera un salto condicional sobre el resultado hacia el final del bucle. Genera el código del bloque y al final agrega un salto incondicional al principio del bucle.
- *ReturnNode* genera código para evaluar la expresión, guardarla en el registro de retorno y saltar incondicionalmente a la subrutina de limpieza y finalización del método.
- *BlockNode* invoca la generación de código de cada una de sus sentencias.
- *SimpleExpressionNode* genera código para evaluar la expresión sin retornar su valor.

ExpressionNode

Cada subclase de *ExpressionNode* debe generar el código propio de ese tipo de expresión:

- *AccessNode* cada una de sus subclases implementa el acceso a la entidad correspondiente del acceso actual, permitiendo su lectura y escritura.
- *ChainedExpressionNode* cada una de sus subclases implementa el acceso al atributo o método correspondiente de la entidad accedida.
- *LiteralNode* genera código para crear en memoria el literal del tipo y valor especificado.
- *UnaryExpressionNode* genera código para evaluar la expresión y realizar la operación elegida, guardando el resultado en el registro de salida.
- *BinaryExpressionNode* genera código para evaluar ambas expresiones y realizar la

operación elegida (realizando las validaciones pertinentes), guardando el resultado en el registro de salida.

Alcance actual y futuros cambios

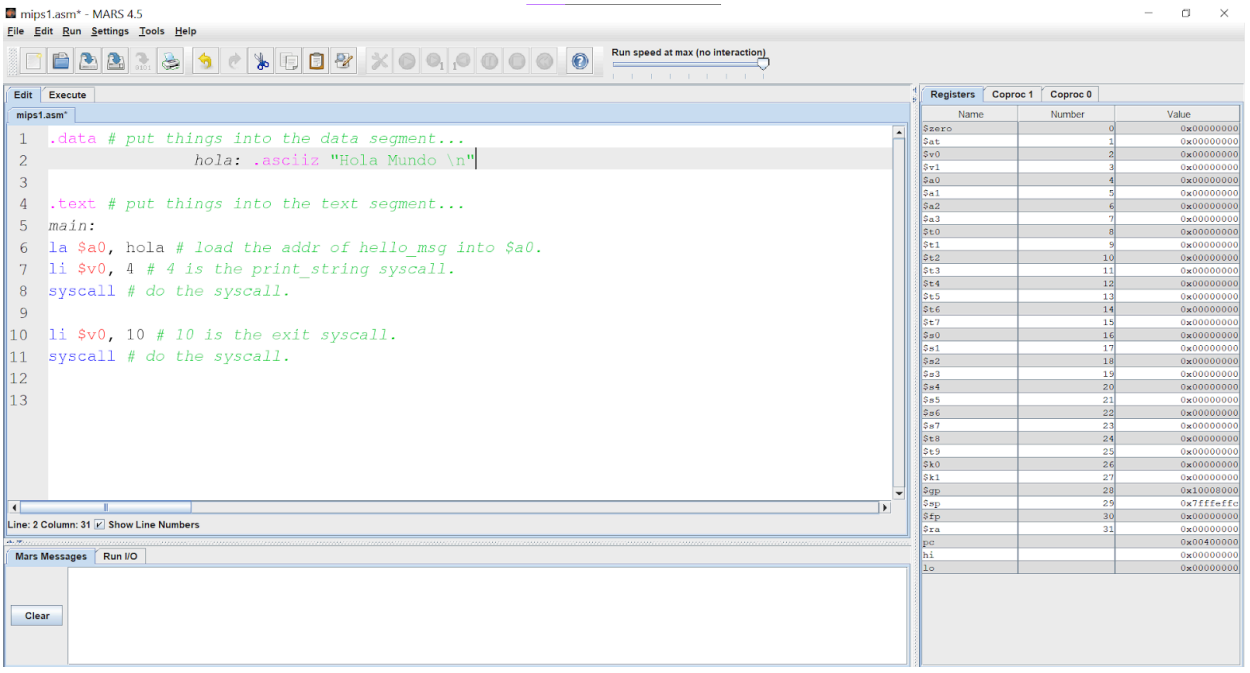
En el estado actual del proyecto, la generación de código aún está incompleta. Si bien se genera el código para los casos de uso más esenciales y básicos del lenguaje, existen numerosas funcionalidades especificadas del lenguaje para las cuales no se ha implementado la generación de código ensamblador. Se planifica completar el desarrollo de estas funcionalidades en un futuro cercano. Se listan estas funcionalidades a continuación:

- Representación y almacenamiento en memoria de arreglos.
- Instanciación y acceso de arreglos.
- Representación y manipulación de datos de tipo *Char*.
- Acceso a atributos y métodos encadenados.
- Métodos predefinidos de las clases *Str* y *Array.*
- Sí se implementaron los métodos predefinidos de *IO in_str*, *in_i32*, *out_str*, *out_bool* y *out_i32*. Queda pendiente la implementación de los demás métodos de esta clase.

Ensamblado y ejecución

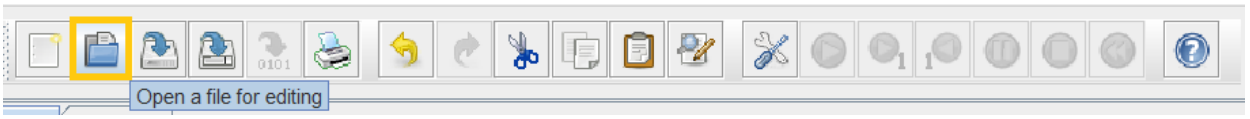
Para la ejecución del código ASM generado por el compilador se debe utilizar algún simulador de la arquitectura MIPS32, como SPIM, MARS o similares. Incluso se pueden encontrar herramientas integradas en los IDE u online diseñadas para este fin. Para dar una mínima noción de esta tarea, se explica cómo realizar el ensamblado y ejecución de los archivos compilados de TinyRust+ utilizando el simulador MARS versión 4.5, pero la tarea se podría realizar igualmente en cualquier otro simulador.

El simulador MARS se puede encontrar en formato JAR ejecutable. Una vez que el programa se ha descargado y abierto, posee una interfaz como la siguiente:

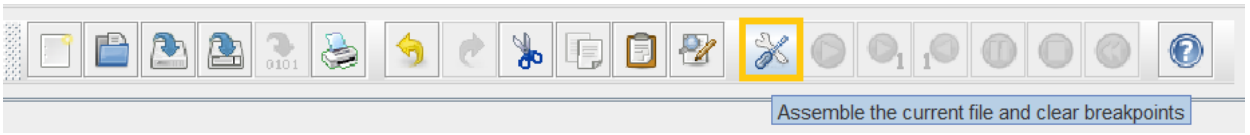


La interfaz posee una barra de acciones en la parte superior y múltiples paneles: en el panel principal se encuentra el código ASM para ejecutar, en el panel lateral derecho se encuentra una lista de todos los registros del procesador (cuyos valores se actualizan en tiempo real durante la ejecución) y en el panel inferior se muestra la salida del ensamblador y la entrada/salida durante la ejecución.

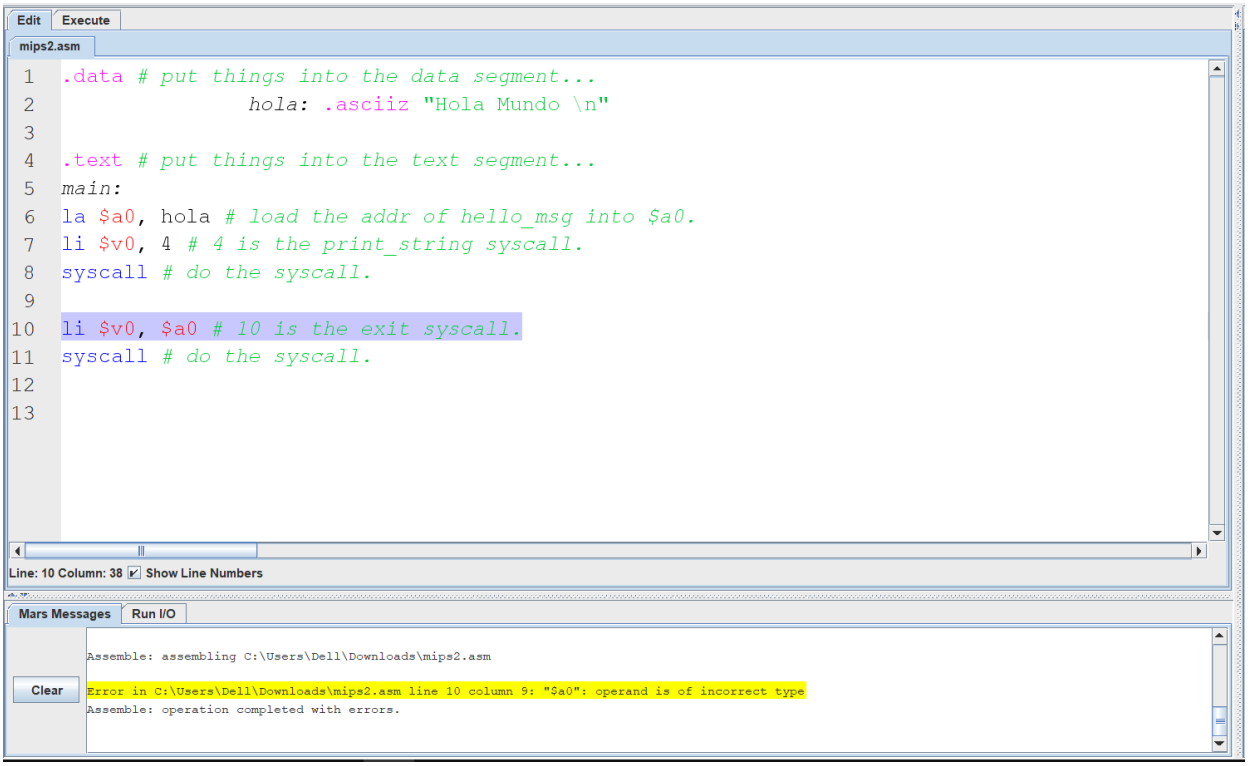
Para comenzar, se debe cargar en el editor el archivo ASM generado por el compilador de TinyRust+. Esto se puede lograr eligiendo la acción “Abrir un archivo” en la barra superior y luego seleccionando el archivo en el selector de archivos emergente.



Luego de cargar el archivo, su código se puede ver en el panel principal. Este código es el equivalente semántico en MIPS32 del código de TinyRust+ original. El siguiente paso es ensamblarlo. Esta acción también se realiza desde el menú superior, eligiendo la acción “Ensamblar archivo actual”.

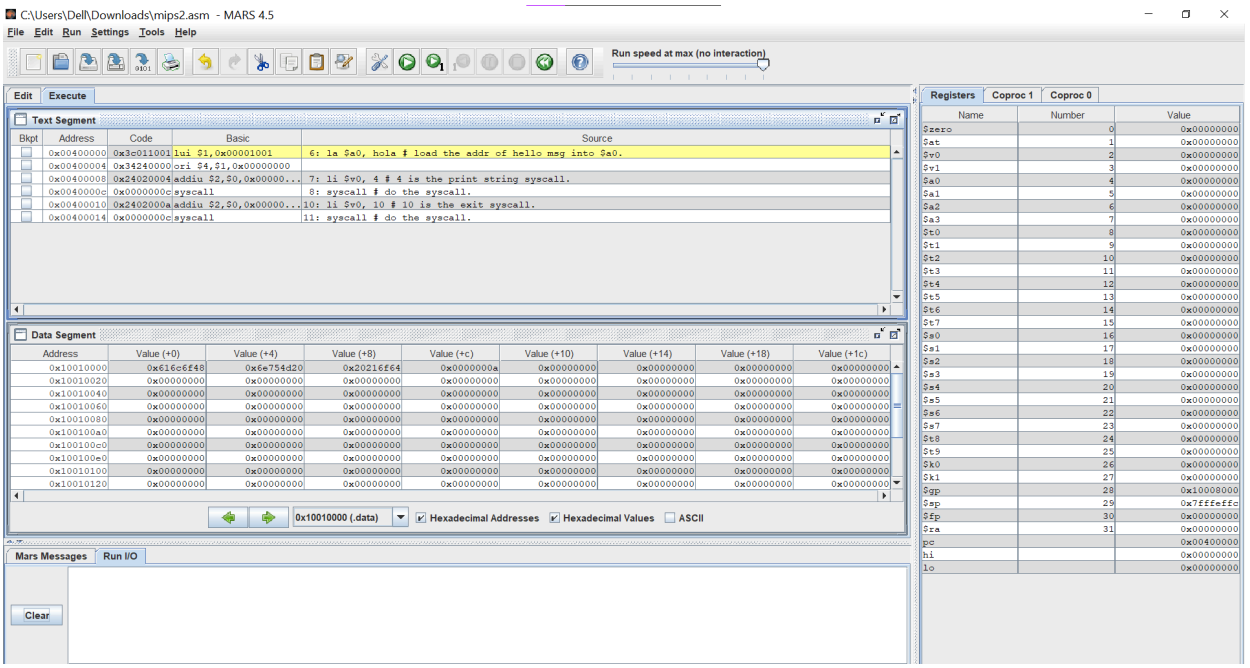


Si el proceso de ensamblado falla, se muestra un error en el panel inferior y en el código fuente se resalta la línea donde ocurrió, como en el siguiente ejemplo:



Se puede modificar el código fuente directamente en el editor de MARS o cargar un nuevo archivo modificado y reintentar el ensamblado siguiendo el mismo procedimiento.

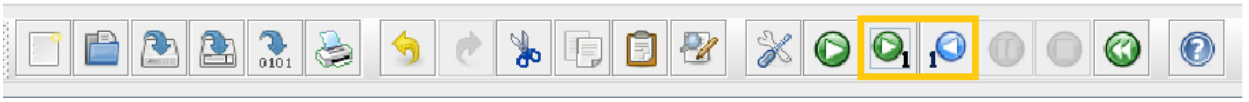
Cuando el código es ensamblado exitosamente, la aplicación se mueve a la pestaña de ejecución. En el panel principal de esta pantalla se pueden visualizar todas las instrucciones MIPS del programa y seguir su ejecución.



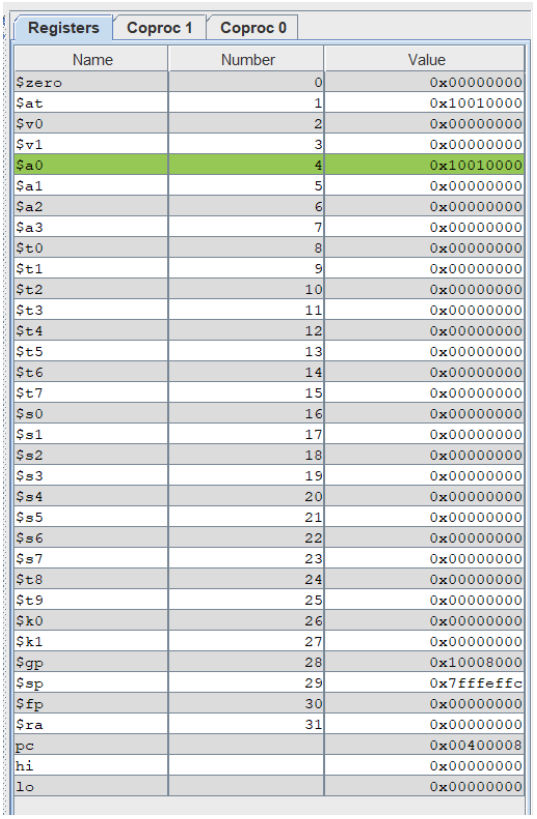
En esta pestaña finalmente se puede ejecutar el código. Se puede ejecutar el programa completo eligiendo la acción “Ejecutar el programa actual” en el menú superior.



Alternativamente, se pueden ejecutar las instrucciones interactivamente una a una utilizando las acciones de avanzar una instrucción y retroceder una instrucción del menú superior. En el panel principal se resaltará en cada paso cuál es la instrucción actual.



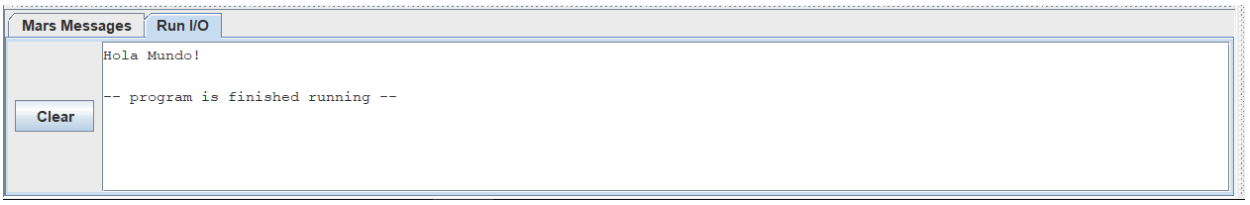
Durante la ejecución, en el panel lateral derecho se puede visualizar en tiempo real el contenido de cada registro. Luego de ejecutar cada instrucción se resalta el registro que fue alterado por esa instrucción (si lo hay).



Registers | Coproc 1 | Coproc 0

Name	Number	Value
$zero	0	0x00000000
$at	1	0x10010000
$v0	2	0x00000000
$v1	3	0x00000000
$a0	4	0x10010000
$a1	5	0x00000000
$a2	6	0x00000000
$a3	7	0x00000000
$t0	8	0x00000000
$t1	9	0x00000000
$t2	10	0x00000000
$t3	11	0x00000000
$t4	12	0x00000000
$t5	13	0x00000000
$t6	14	0x00000000
$t7	15	0x00000000
$s0	16	0x00000000
$s1	17	0x00000000
$s2	18	0x00000000
$s3	19	0x00000000
$s4	20	0x00000000
$s5	21	0x00000000
$s6	22	0x00000000
$s7	23	0x00000000
$t8	24	0x00000000
$t9	25	0x00000000
$k0	26	0x00000000
$k1	27	0x00000000
$gp	28	0x10008000
$sp	29	0x7fffeffc
$fp	30	0x00000000
$ra	31	0x00000000
pc		0x00400008
hi		0x00000000
lo		0x00000000

La entrada y salida se verá en el panel inferior de la pantalla, donde se puede leer y escribir siguiendo el flujo del programa original. En caso de ocurrir un error en tiempo de ejecución, aparecerá en este panel. Cuando se completa la ejecución del archivo, se imprime en este medio un mensaje notificando. Para el archivo de ejemplo *holaMundo.rs*, la salida es la siguiente:



Ejemplo

Un ejemplo básico con el cual se puede probar el funcionamiento del compilador es la siguiente implementación de Fibonacci (con ligeras modificaciones del archivo original). Este archivo genera una salida *ASM* de aproximadamente 800 líneas que puede ser ejecutada en cualquiera de los simuladores de MIPS32.

```
class Fibonacci {
    I32: suma;
    I32: i, j;

    fn sucesion_fib(I32: n) -> void {
      i=0; j=0; suma=0;
      while (i<= n) {
```

```
            if (i==0) {
                  (imprimo_numero(i));
                  (imprimo_sucesion(suma));
            } else if (i==1) {
                  (imprimo_numero(i));
                  suma=suma+i;
                  (imprimo_sucesion(suma));
            } else {
                  (imprimo_numero(i));
                  suma=suma+j;
                  j=suma;
                  (imprimo_sucesion(suma));
            }
            i = i + 1;
      }
    }

    create(I32: n) {
      i=0;
      j=0;
      suma=0;
      (sucesion_fib(n));
    }

    fn imprimo_numero(I32: num) -> void {
      (IO.out_str("f_"));
      (IO.out_i32(num));
      (IO.out_str("="));
    }

    fn imprimo_sucesion(I32: s) -> void {
      //"el valor es: ";
      (IO.out_i32(s));
      (IO.out_str("\n"));
    }
}

fn main() {
    Fibonacci: fib;
    I32: n;
    n = IO.in_i32();
    fib = new Fibonacci(n);
}
```